Supplementary Material to Two-Time-Scale Transfer Learning for Market-by-Order Micro-Return Forecasting

Juan Diego Mejia Becerra[1], Gareth W. Peters[1], and Ruimeng Hu[1,2]

[1]Department of Statistics and Applied Probability, University of California, Santa Barbara
[2]Department of Mathematics, University of California, Santa Barbara

Last Modified: February 10, 2026

Contents

A Mathematical Representation of the Order Flow Sequence and Limit Order Book

This section is devoted to mathematically describing the OF sequence and the LOB drawing ideas from **???**. First observe that most LOBs operate with a minimum price variation, or tick size, denoted θ, and a maximum price, denoted D. For simplicity, and without losing generality, prices will be measured in ticks. Also let $d = \frac{D}{\theta} \in \mathbb{N} \setminus \{0\}$ be the maximum price measured in ticks and denote $\mathbb{N}_{0:d} = \{0, 1, \dots, d\}$ the set of possible tick-measured prices.

Definition 1. *Let $o = (p_o, z_o, t_o, i_o, s_o, k_o)$ denote a message submitted to the matching engine at time $t_o \in [0, T]$, with price $p_o \in \mathbb{N}_{0:d}$, size $z_o \in \mathbb{N}$, identifier $i_o \in \mathbb{N}$, side $s_o \in \{1, -1\}$ and type $k_o \in \{L, C, M\}$. $s_o = 1$ ($s_o = -1$) indicates that the message o is submitted in the bid (ask) side of the LOB. L, C and M identify limit, cancellations and market orders respectively.*

Definition 2. *Let $\mathbf{o} = \{o_i\}_{i=1}^N$, $N \in \mathbb{N}$, denote an ordered sequence of messages such that $t_{o_i} \leq t_{o_{i+1}}$. This sequence is known as the order flow sequence.*

To favor notation lightness, The subscript o_i will be dropped in favor of i in the elements of the message sequence $\mathbf{o}$ whenever no ambiguity is present (e.g. t_i instead of t_{o_i} will be used). For simplicity, identify buy (sell) market orders in the order sequence $\mathbf{o}$ as limit orders with the maximum (minimum) possible price. That is,

$$(p_o, z_o, t_o, i_o, 1, M) \equiv (d, z_o, t_o, i_o, 1, L)$$
$$(p_o, z_o, t_o, i_o, -1, M) \equiv (0, z_o, t_o, i_o, -1, L)$$

Definition 3. *Let $d \in \mathbb{N} \setminus \{0\}$ be the maximum price of an asset. The order aggregator, $\mathcal{X} : \mathbb{N} \times \mathbb{N} \times [0, T] \times \mathbb{N} \times \{-1, 1\} \times \{L, C\} \to \mathbb{Z}^d \times \mathbb{Z}^d$, groups limit orders and cancellations by price and adds its sizes*

$$\mathcal{X}(\mathbf{o}) = \left(\sum_{o \in \mathbf{o}} (I_L^+(o) - I_C^+(o)) z_o e_{p_o}, \sum_{o \in \mathbf{o}} (I_L^-(o) - I_C^-(o)) z_o e_{p_o} \right)$$

Where the market orders in $\mathbf{o}$ are identified as limit orders, $e_i = (0, 0, \dots, 0, 1, 0, \dots, 0) \in \mathbb{Z}^d$ where 1 is the i-th position, $I_k^+(o)$ ($I_k^-(o)$) takes value 1 if $s_o = 1$ ($s_o = -1$) and $k_o = k$, $k \in \{L, C\}$.

Two pathological scenarios might arise with the established notation. First, a cancellation message might exceed the size of the initial order or target a nonexistent order, inadvertently transforming into a market order that could circumvent the established price-time priority rules. Second, a single order could deplete all available liquidity on the opposite side of the LOB, leading to a situation where the price becomes undefined. Although these scenarios are rare, occurring under highly specific conditions, we preclude them by setting forth two basic assumptions:

(A1) Every cancellation correspond to a message submitted earlier, with no cancellation exceeding the outstanding size

(A2) No buy (sell) market order exceeds the ask (bid) volume of the limit order book.

The set of possible values of $\mathbf{o}$ under the restrictions (A1) and (A2) is denoted as $\mathcal{O}$. To reconstruct the LOB, the modular framework described in **?** is employed. This framework decomposes the dynamics of the limit order book into two distinct elements: the incoming order flow and the market clearing procedure. The incoming order flow is conceptualized as a marked point process, while the market clearing procedure is a deterministic operator responsible for pairing buy and sell orders.

Denote $X = (X^+, X^-) \in E := \mathbb{N}^d \times \mathbb{N}^d$ a configuration of limit orders.[1] Observe that the set of valid configurations of LOBs is a proper subset of E. Some notation is necessary to determine this set. First, for $Z \in \mathbb{N}^d$, consider the measure

$$\nu_Z = \sum_{k=1}^d Z_k \delta_k.$$

Where δ_k denotes the Dirac's delta measure at $k \in \{1, \dots, d\}$. Also, let

[1] i.e. $X = \mathcal{X}(\mathbf{x})$ for some $\mathbf{x} \in \mathcal{O}$.

- $\nu_X^{\pm} = \nu_{X^{\pm}}$
- $\tau^i(Z)$ be such that $\nu_{\tau^i(Z)} = \nu_Z 1_{[i,\infty)}$
- $\tau_i(Z)$ be such that $\nu_{\tau_i(Z)} = \nu_Z 1_{[0,i]}$
- $a(X) = \inf \operatorname{supp}(\nu_X^+)$, with $a(0) = 0$
- $b(X) = \sup \operatorname{supp}(\nu_{X^+}^-)$, with $b(0) = d$

Observe that $a(X)$ and $b(X)$ correspond to the best ask and bid of a limit order book snapshot respectively. The set of valid limit order book configurations is thus

$$\mathcal{L} := \{X = (X^+, X^-) \in E : a(X) > b(X)\}$$

Definition 4. *Let $\mathbf{o} \in \mathcal{O}$. Define the mapping $J : [0,T] \times \mathcal{O} \to \mathbb{Z}^d \times \mathbb{Z}^d$ as follows*

$$J(t, \mathbf{o}) = \begin{cases} \mathcal{X}(\{o \in \mathbf{o} : t_o = t_i\}) & \text{if } t = t_i \ \exists i = 1, \dots, N, \\ 0 & \text{otherwise.} \end{cases}$$

Observe that J models changes in the LOB right before matching. The matching process is modelled in **?** through a deterministic order matching clearing operator $\mathcal{C} : E \to \mathcal{L}$ whose set of invariant points is $\mathcal{L}$. It can be shown that, under some conditions, $\mathcal{C}$ is uniquely characterized as follows:

$$\mathcal{C}(X)^+ = \begin{cases} \tau_{p_B(X)}(X^+) & \text{if } g_X(p_B(X)) \leq -X^+_{p_B(X)}, \\ \tau_{p_B(X)-1}(X^+) - \min(0, g_X(p_B(X)))e_{p_B}(X) & \text{if } g_X(p_B(X)) > -X^+_{p_B(X)}. \end{cases}$$

$$\mathcal{C}(X)^- = \begin{cases} \tau^{p_A(X)}(X^+) & \text{if } g_X(p_A(X)) \geq X^-_{p_A(X)}, \\ \tau^{p_A(X)-1}(X^+) + \max(0, g_X(p_A(X)))e_{p_A}(X) & \text{if } g_X(p_A(X)) < X^-_{p_A(X)}. \end{cases}$$

Where $X \in E$ is the set of limit orders. Let $B_X(k) = \nu^+[k, \infty)$, $S_X(k) = \nu_X^-[0,k]$, $g(k) = S_X(k) - B_X(k)$, and

$$p_A(X) = \inf\{k \in \{1, \dots, d\} : g_X(k) > 0\},$$
$$p_B(X) = \sup\{k \in \{1, \dots, d\} : g_X(k) < 0\}.$$

The reconstruction of an LOB given $\mathbf{o} \in \mathcal{O}$ is now straightforward:

Definition 5. *Given $\mathbf{o} \in \mathcal{O}$, the reconstructed LOB is a mapping $L : [0,T] \times \mathcal{O} \to \mathcal{L}$ such that*

$$L(t, \mathbf{o}) = \begin{cases} L(t_*, \mathbf{o}) & \text{if } t \neq t_i \ \forall i = 1, \dots, N, \\ \mathcal{C}\left(L(t_*, \mathbf{o}) + J(t_i, \mathbf{o})\right) & \text{if } t = t_i \ \exists i = 1, \dots, N. \end{cases}$$

Here, $t_* = \max\{t_j : t_j < t\}$.

Definition 6. *For a valid LOB configuration, $X \in \mathcal{L}$, let $b_h(X) = b(X) - h$ ($a_h(X) = a(X) + h$) be the price h ticks below (above) the best bid (ask). Also let $V_h^b(X) = \nu_X^+\{b_h(X)\}$ ($V_h^a(X) = \nu_X^-\{a_h(X)\}$) be the corresponding volume sitting h ticks below (above) the best ask. For convenience, denote $a_h(t) = a_h(X(t))$, $b_h(t) = b_h(X(t))$, $V_h^a(t) = V_h^a(X(t))$ and $V_h^b(t) = V_h^b(X(t))$.*

An alternative approach, known as the compressed representation, to describe the liquidity available in the LOB is in terms of its levels. The l-th bid (ask) price level is defined as the l-th highest (lowest) price in the LOB that has positive volume available. Formally,

Definition 7. *For a valid LOB configuration, $X \in \mathcal{L}$, the l-th level bid (ask) price, is defined iteratively as $p_l^b(X) = \inf \operatorname{supp}\left(\tau_{p_{l-1}^b(X)-1}(X^+)\right)$ ($p_l^a(X) = \inf \operatorname{supp}\left(\tau^{p_{l-1}^a(X)+1}(X^-)\right)$) for $l \geq 1$ and $p_0^b(X) = b_0(X)$ ($p_0^a(X) = a_0(X)$). Similarly, denote the volume available at the l-th bid (ask) level as $v_l^b(X) = \nu_X^+\{p_l^b(X)\}$ ($v_l^a(X) = \nu_X^-\{p_l^a(X)\}$). To keep notation light, denote $p_l^a(t) = p_l^a(X(t))$, $p_l^b(t) = p_l^b(X(t))$, $v_l^a(t) = v_l^a(X(t))$ and $v_l^b(t) = v_l^b(X(t))$.*

Each representation offers distinct trade-offs. For instance, working with decompressed limit order book data may simplify the mathematical modeling of order flow, while compressed formats are more memory-efficient—an especially important advantage for illiquid assets. Moreover, compressed representations often provide a more suitable input to neural network models, potentially enhancing their predictive accuracy.

We have assumed a fixed message sequence $\mathbf{o} \in \mathcal{O}$ and reconstructed the limit order book for such sequence. To add stochasticity, let $(\Omega, \mathcal{F}, \mathbb{F}, \mathbb{P})$ be a filtered, complete and right-continuous probability space and consider the following

Definition 8. *Let τ be a $\mathbb{F}$-stopping time and π, ζ, ι, σ and κ; $\mathcal{F}_\tau$-adapted random variables with values in $\mathbb{N}_{0:d}$, $\mathbb{N}$, $\mathbb{N}$, $\{-1, 1\}$, and $\{L, C, M\}$ respectively. The corresponding stochastic message is denoted as $\omega = (\pi, \zeta, \tau, \iota, \sigma, \kappa)$.*

Analogously, for a sequence of non-explosive increasing sequence of $\mathbb{F}$-adapted stopping times $(\tau_i)_{i=1}^{\infty}$ and $\mathcal{F}_{\tau_i}$-measurable random variables π_i, ω_i, σ_i, ι_i and κ_i for which assumptions (A1) and (A2) hold, define the stochastic message sequence

$$\boldsymbol{\omega} = (\omega_i : \tau_i \leq T).$$

Where, $\omega_i = (\pi_i, \zeta_i, \tau_i, \iota_i, \sigma_i, \kappa_i)$.

The L function from definition **??** can be employed to specify a stochastic LOB.

Definition 9. *The LOB process is a stochastic process, denoted $(X(t))_{t\geq 0}$ and defined as*

$$X(t) = L(t, \boldsymbol{\omega}).$$

For the sake of clarity, the previous definitions are illustrated below. Consider the following order flow sequence $\mathbf{o} \in \mathcal{O}$ represented in the following table

p_i	z_i	t_i	i_i	s_i	k_i
10	10	0.04	1	1	L
11	20	1.23	2	1	L
12	100	2.45	3	-1	L
12	50	3.14	4	-1	L
8	50	4.75	5	-1	L
9	150	5.63	6	1	L
13	30	6.14	7	1	L
13	45	7.85	8	-1	L
12	70	8.32	3	-1	C
12	50	9.78	4	-1	C

Table 1: Example OF sequence

Definition **??** applied to the sequence $\mathbf{o}$ implies

$$\begin{aligned}\mathcal{X}(\mathbf{o}) = \big(&150e_9 + 10e_{10} + 20e_{11} + 30e_{13},\\ &50e_8 + 30e_{12} + 45e_{13}\big).\end{aligned}$$

This object is depicted in Figure **??**.

Following the price-time priority rule, orders 1 through 4 are not marketable and are simply added to the LOB. The aggressive sell order 5 then arrives, clearing orders 1 and 2 entirely and leaving 20 units unexecuted on the ask side sitting at 8 ticks, with the bid side empty. When the buy order 6 arrives, it clears the remaining 20 units of order 5, leaving 130 units on the bid side sitting at 9 ticks. Order 7 next clears 30 units of order 3, leaving 120 units at 12 ticks on the ask side. Another 45 units are then added at price 13 on the ask side. Finally, messages 9 and 10 cancel the remaining volumes of orders 3 and 4. Thus leaving 130 units sitting on the bid side at 9 ticks and 45 units sitting at 13 ticks on the ask side. As a side note, observe that $\mathcal{C}\left(\mathcal{X}(\mathbf{o})\right) = (150e_9 + 10e_{10}, 30e_{12} + 45e_{13})$ which is different from the resulting



Figure 1: Aggregated example orders.

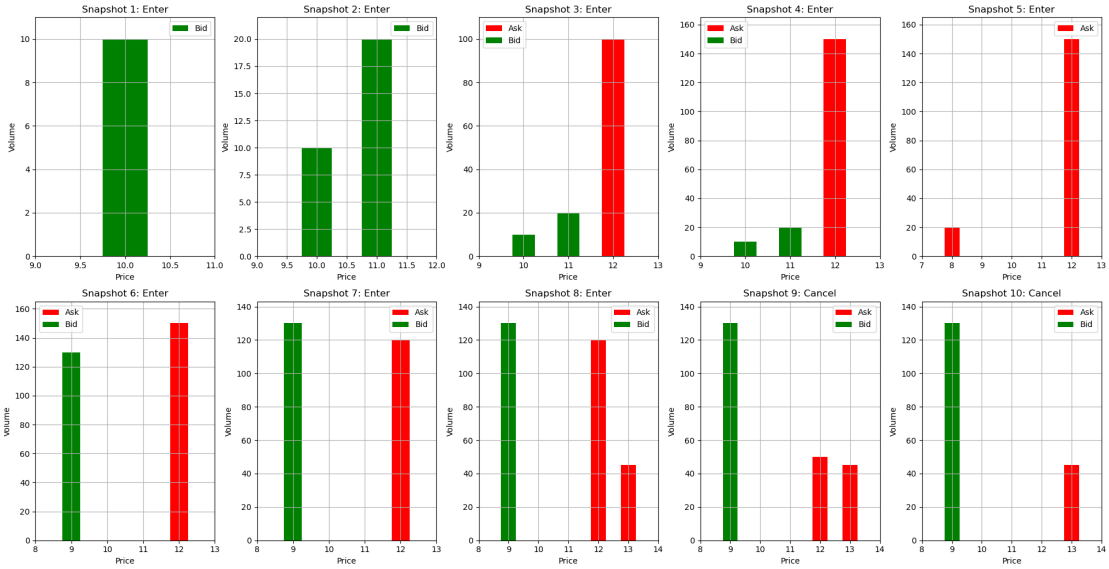


Figure 2: Sequence of LOB snapshots corresponding to table **??**.

LOB of the example $L(t = 10, \mathbf{o}) = (130e_9, 45e_{13})$. This reflects the fact that continuously clearing the LOB (as in trading hours) can yield different results to discrete clearing (as in opening/closing auctions).

Figure **??** illustrates the corresponding sequence of LOB snapshots resulting from these events.

A useful distinction between the OF sequence and MBO data lies in how MBO data can be constructed from the OF sequence without requiring explicit order matching. One way to achieve this is by labeling addition messages as `A` and cancellation messages as `C`. Whenever an outstanding order executes, a series of messages—`T` (trade), `F` (filled), and `C` (cancellation)—can be inserted at the same timestamp to indicate the trade initiator, the filled order, and the final cancellation needed for LOB reconstruction. Under this scheme, the LOB can be fully reconstructed using only `A` and `C` messages, eliminating the need for a separate clearing process. Table **??** illustrates this procedure.

timestamp	order_id	side	volume	price	message_type
0.04	1	B	10	10	A
1.23	2	B	20	11	A
2.45	3	A	100	12	A
3.14	4	A	50	12	A
4.75	5	A	20	11	T
4.75	2	B	20	11	F
4.75	2	B	20	11	C
4.75	5	A	10	10	T
4.75	1	B	10	10	F
4.75	1	B	10	10	C
4.75	5	A	20	8	A
5.63	6	B	20	8	T
5.63	5	A	20	8	F
5.63	5	A	20	8	C
5.63	6	B	130	9	A
6.14	7	B	30	12	T
6.14	3	A	30	12	F
6.14	3	A	30	12	C
7.85	8	A	45	13	A
8.32	3	A	70	12	C
9.78	4	A	50	12	C

Table 2: Example MBO data.

B Alternative Sampling Schemes

For completeness, we define the *volume bars* partition $\mathcal{P}^{(V)}$ via the stopping times

$$\tau_i^{(V)} \;=\; \inf\Big\{\, t > \tau_{i-1}^{(V)} : \sum_{k:\,\kappa_k = M} \zeta_k > i \cdot \texttt{Vol} \Big\}, \quad \tau_0^{(V)} = 0,$$

where $i, \texttt{Vol} \in \mathbb{Z}^+$. Each interval $\Delta\tau_i^{(V)} \;=\; \tau_i^{(V)} - \tau_{i-1}^{(V)}$ corresponds to the time required for `Vol` units to trade. Furthermore, given any partition $\mathcal{P}$, we define the counting process

$$N^{\mathcal{P}}(t) \;=\; \Big|\big\{\tau \in \mathcal{P} :\, 0 < \tau \le t\big\}\Big|,$$

such that $N^{\mathcal{P}}(t_1, t_2) = N^{\mathcal{P}}(t_2) - N^{\mathcal{P}}(t_1)$ counts the number of partition points in $(t_1, t_2]$.

C CNN–LSTM and DeepLOB Model Specifications

This section formalizes the CNN–LSTM and DeepLOB architectures **??**. Section **??** introduces the fundamental tensor operations, which are assembled into the composite layers described in Section **??**. These composite layers are then connected via convolutional and inception modules to yield the full CNN–LSTM and DeepLOB architectures, as detailed in Section **??**.

C.1 Building Blocks

This section describes the fundamental tensor operations that compose the CNN-LSTM and DeepLOB architectures.

C.1.1 Permutation

For simplicity, consider a tensor $T \in \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d}$ as a multidimensional array with entries denoted by $T_{i_1 i_2 \cdots i_d}$. Denote the data batches as four-dimensional tensors in $\mathbb{R}^{N \times C \times H \times W}$, where N denotes the batch size, C the number of channels, H the temporal length, and W the spatial width.

Tensor permutation serves as a higher-dimensional analogue to the matrix transpose. Define the permutation between the k-th and l-th dimensions as

$$\begin{aligned} \mathrm{Perm}^{k,l} : \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_k \times \cdots \times N_l \times \cdots \times N_d} &\to \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_l \times \cdots \times N_k \times \cdots \times N_d} \\ T &\mapsto \mathrm{Perm}^{k,l}(T) \end{aligned}$$

where $\left[\mathrm{Perm}^{k,l}(T)\right]_{i_1 i_2 \cdots i_k \cdots i_l \cdots i_d} = T_{i_1 i_2 \cdots i_l \cdots i_k \cdots i_d}$.

C.1.2 Concatenation

Tensor concatenation along the k-th dimension is defined as

$$\begin{aligned} \mathrm{Concat}^k : \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_k \times \cdots \times N_d} \times \mathbb{R}^{N_1 \times N_2 \times \cdots \times N'_k \times \cdots \times N_d} &\to \mathbb{R}^{N_1 \times N_2 \times \cdots \times (N_k + N'_k) \times \cdots \times N_d} \\ (T, S) &\mapsto \mathrm{Concat}^k(T, S) \end{aligned}$$

where,

$$\left[\mathrm{Concat}^k(T, S)\right]_{i_1 i_2 \cdots i_k \cdots i_d} = \begin{cases} T_{i_1 i_2 \cdots i_k \cdots i_d} & 1 \le i_k \le N_k, \\ S_{i_1 i_2 \cdots i_k \cdots i_d} & N_k + 1 \le i_k \le N_k + N'_k. \end{cases}$$

C.1.3 Activation Functions

Two activation functions are necessary: The Sigmoid and the Leaky RelU. For a scalar $x \in \mathbb{R}$ these two activation functions are defined as

$$\begin{aligned} \mathrm{Sigmoid}(x) &= \frac{1}{1 + \exp(-x)}, \\ \mathrm{LeakyReLU}_\alpha(x) &= \begin{cases} x & x \ge 0, \\ \alpha x & x < 0. \end{cases} \end{aligned}$$

For a tensor $T \in \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d}$, define $\mathrm{Sigmoid} : \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d} \to \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d}$ and $\mathrm{LeakyReLU}_\alpha : \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d} \to \mathbb{R}^{N_1 \times N_2 \times \cdots \times N_d}$

$$\begin{aligned} \left[\mathrm{Sigmoid}(T)\right]_{i_1 i_2 \cdots i_d} &= \mathrm{Sigmoid}(T_{i_1 i_2 \cdots i_d}), \\ \left[\mathrm{LeakyReLU}_\alpha(T)\right]_{i_1 i_2 \cdots i_d} &= \mathrm{LeakyReLU}_\alpha(T_{i_1 i_2 \cdots i_d}). \end{aligned}$$

C.1.4 Batch Normalization

In the CNN-LSTM model, Spatial Batch Normalization, PyTorch's `BatchNorm2d`[2], originally proposed in **?**, is applied to each convolutional feature map. For a tensor of size N and feature maps of shape (C, H, W), `BatchNorm2d` computes per-channel statistics over all $N \times H \times W$ activations, then learns one pair of affine parameters (γ_c, β_c). This normalization is supposed to alleviate the internal covariate shift problem by fixing each channel's mean to zero and variance to one. This stabilization has been empirically shown to allow for higher learning rates and faster convergence, and can indirectly alleviate issues like vanishing or exploding gradients.

More precisely, let $T \in \mathbb{R}^{N\times C\times H\times W}$ denote an activation tensor of size N each with C channels of spatial size $H \times W$. Denote by T_{icjk} the element for sample i, channel c, and spatial position (j, k). The Spatial Normalization operation is an application $BN^{\phi,\varepsilon} : \mathbb{R}^{N\times C\times H\times W} \to \mathbb{R}^{N\times C\times H\times W}$ defined as

$$\big[\mathrm{BN}^{\phi,\varepsilon}(T)\big]_{icjk} = \gamma_c \frac{T_{icjk} - \mu_c}{\sqrt{\sigma_c^2 + \varepsilon}} + \beta_c.$$

Here, γ_c and β_c are trainable parameters, μ_c and σ_c^2 are channel means and variances respectively, ϕ is a momentum hyperparameter and, ε is a constant used for numerical stability. The computations of μ_c and σ_c^2 differ for training and testing. During training, these quantities are computed in a batch-wise fashion as

$$\mu_c = \mu_c^{(B)} = \frac{1}{N\,H\,W}\sum_{i=1}^{N}\sum_{j=1}^{H}\sum_{k=1}^{W} T_{icjk}, \quad \sigma_c^2 = \sigma_c^{2\,(B)} = \frac{1}{N\,H\,W}\sum_{i=1}^{N}\sum_{j=1}^{H}\sum_{k=1}^{W}\big(T_{icjk} - \mu_c^{(B)}\big)^2.$$

During testing (or inference) the batch mean, these quantities are computed in the likes of a exponential moving average with momentum ϕ. That is

$$\mu_c \leftarrow (1-\phi)\,\mu_c + \phi\,\mu_c^{(B)}, \qquad \sigma_c^2 \leftarrow (1-\phi)\,\sigma_c^2 + \phi\,\sigma_c^{2\,(B)}.$$

The interested reader is invited to consult **?** for details on the gradients of this transformation and some numerical experiments.

C.1.5 Max-Pooling

Max-pooling is a downsampling operation that accentuates the strongest activations within local neighborhoods. Given a 4D tensor $T \in \mathbb{R}^{N\times C\times H\times W}$, a max-pool layer with window size $n_h \times n_w$ and stride (s_h, s_w) produces an output tensor

$$\mathrm{MP}^{n_h,n_w}_{s_h,s_w}(T) \in \mathbb{R}^{N\times C\times H'\times W'},$$

where

$$H' = \left\lfloor \frac{H - n_h}{s_h} \right\rfloor + 1, \quad W' = \left\lfloor \frac{W - n_w}{s_w} \right\rfloor + 1.$$

Its entries are given by

$$\big[\mathrm{MP}^{n_h,n_w}_{s_h,s_w}(T)\big]_{ickl} = \max\{T_{ick'l'} : k' \in \{(k-1)s_h + i' : i' = 1, 2, \ldots, n_h\}, \\ l' \in \{(l-1)s_w + j' : j' = 1, 2, \ldots, n_w\}\},$$

for $i = 1, \ldots, N$, $c = 1, \ldots, C$, $k = 1, \ldots, H'$, $l = 1, \ldots, W'$. In other words, each output cell $\Big[\mathrm{MP}^{n_h,n_w}_{s_h,s_w}(T)\Big]_{ickl}$ is the maximum value within the $n_h \times n_w$ patch of T starting at spatial location $((k-1)\,s_h, (l-1)\,s_w)$ in channel c of sample i.

C.1.6 Zero-Padding

Convolution and pooling layers typically reduce spatial dimensions. However, certain architectures require the output tensor to have the same height and width as the input. The Zero-padding operation

[2] `https://pytorch.org/docs/stable/generated/torch.nn.BatchNorm2d.html` (Accessed on 04-29-2025)

achieves this by surrounding the input with zeros along its spatial axes. Formally, for $T \in \mathbb{R}^{N\times C\times H\times W}$, the padded tensor

$$\mathrm{ZP}_{p_h,p_w}(T) \in \mathbb{R}^{N\times C\times(H+2p_h)\times(W+2p_w)}$$

has entries

$$\big[\mathrm{ZP}_{p_h,p_w}(T)\big]_{ickl} = \begin{cases} 0, & k \le p_h \quad \text{or} \quad k > H + p_h, \\ 0, & l \le p_w \quad \text{or} \quad l > W + p_w, \\ T_{ic(k-p_h)(l-p_w)}, & \text{otherwise}, \end{cases}$$

for $i = 1,\dots,N$, $c = 1,\dots,C$, $k = 1,\dots,H+2p_h$, and $l = 1,\dots,W+2p_w$. This adds p_h rows of zeros above and below, and p_w columns of zeros to the left and right of each feature map.

C.1.7 Fully Connected Layer

A fully connected layer performs an affine transformation of its input. Concretely, given an input vector $a \in \mathbb{R}^D$, a weights matrix $W^{(\mathrm{FC})} \in \mathbb{R}^{n_h\times D}$, and a bias vector $b \in \mathbb{R}^{n_h}$, the layer output is

$$\mathrm{FC}_{n_h}(a) \;=\; W^{(\mathrm{FC})}\,a \;+\; b \;\in\; \mathbb{R}^{n_h}.$$

When applied to a batch of N inputs, arranged as $A \in \mathbb{R}^{N\times D}$, the same transformation is applied independently to each row:

$$\big[\mathrm{FC}_{n_h}(A)\big]_{i\cdot} \;=\; \mathrm{FC}_{n_h}\big(A_{i\cdot}\big), \quad i = 1,\dots,N.$$

Note that $\mathrm{FC}_{n_h}(A) \in \mathbb{R}^{N\times n_h}$

C.1.8 Convolutional Filters

Convolutional filters detect local patterns—such as edges or textures—by sliding small learnable kernels over the input. Concretely, let $T \in \mathbb{R}^{N\times C\times H\times W}$ and $W^{(\mathrm{Conv})} \in \mathbb{R}^{C'\times C\times k_h\times k_w}$ denote, respectively, a batch of N feature-map tensors with C channels and spatial size $H \times W$, and a set of C' convolutional kernels of size $k_h \times k_w$. With stride (s_h, s_w) and bias vector $b \in \mathbb{R}^{C'}$, the convolution operation produces the output

$$\big[\mathrm{Conv}^{C',k_h,k_w}_{s_h,s_w}(T)\big]_{ic'kl} = \big[T * W^{(\mathrm{Conv})}\big]_{ic'kl} + b_{c'},$$

where

$$\big[T * W^{(\mathrm{Conv})}\big]_{ic'kl} = \sum_{c=1}^{C}\sum_{k'=1}^{k_h}\sum_{l'=1}^{k_w} T_{ic((k-1)s_h+k')((l-1)s_w+l')} W^{(\mathrm{Conv})}_{c'ck'l'}$$

for $i = 1,\dots,N$, $c' = 1,\dots,C'$, $k = 1,\dots,H'$, and $l = 1,\dots,W'$. Note that $\mathrm{Conv}^{C',k_h,k_w}_{s_h,s_w}(T) \in \mathbb{R}^{N\times C'\times H'\times W'}$ where

$$H' \;=\; \left\lfloor \frac{H-k_h}{s_h} \right\rfloor + 1, \quad W' \;=\; \left\lfloor \frac{W-k_w}{s_w} \right\rfloor + 1.$$

If the stride is $(1,1)$, the subscript may be omitted for notational simplicity.

C.1.9 LSTM Layer

A long–short term memory (LSTM) layer processes sequential inputs while maintaining internal cell and hidden states. Define the LSTM operator H_{n_h} for input dimension D and hidden dimension n_h as follows. For $x \in \mathbb{R}^D, \quad a, c \in \mathbb{R}^{n_h}$, set

$$H_{n_h}(x, a, c) = (a', c').$$

Where

$$\begin{aligned}
\Gamma_f &= \mathrm{Sigmoid}\big(W^{(fx)}\,x + W^{(fa)}\,a + b^{(f)}\big),\\
\Gamma_u &= \mathrm{Sigmoid}\big(W^{(ux)}\,x + W^{(ua)}\,a + b^{(u)}\big),\\
\Gamma_o &= \mathrm{Sigmoid}\big(W^{(ox)}\,x + W^{(oa)}\,a + b^{(o)}\big),\\
\tilde{c} &= \tanh\big(W^{(cx)}\,x + W^{(ca)}\,a + b^{(c)}\big),\\
c' &= \Gamma_f \odot c + \Gamma_u \odot \tilde{c},\\
a' &= \Gamma_o \odot \tanh(c').
\end{aligned}$$

Here, for each gate symbol $\star \in \{f, u, o, c\}$,

$$W^{(\star x)} \in \mathbb{R}^{n_h \times D}, \quad W^{(\star a)} \in \mathbb{R}^{n_h \times n_h}, \quad b^{(\star)} \in \mathbb{R}^{n_h},$$

are trainable parameters, and $\odot$ denotes element-wise product. Given a sequence $T \in \mathbb{R}^{H \times D}$ with rows $x^{\langle i \rangle}$ for $i = 1, \ldots, H$, initialize

$$a^{\langle 0 \rangle} = 0, \quad c^{\langle 0 \rangle} = 0,$$

and recursively compute for $i = 1, \ldots, H$,

$$(a^{\langle i \rangle}, c^{\langle i \rangle}) = H_{n_h}\big(x^{\langle i \rangle}, a^{\langle i-1 \rangle}, c^{\langle i-1 \rangle}\big).$$

Define the sequence-to-vector transformation

$$\begin{aligned}
\mathrm{LSTM}_{n_h} : \mathbb{R}^{H \times D} &\to \mathbb{R}^{n_h},\\
T &\mapsto \mathrm{LSTM}_{n_h}(T) = a^{\langle H \rangle}.
\end{aligned}$$

For batched input $T \in \mathbb{R}^{N \times H \times D}$, the LSTM acts independently on each batch slice:

$$\big[\mathrm{LSTM}_{n_h}(T)\big]_{i\cdot} = \mathrm{LSTM}_{n_h}(T_{i\cdot\cdot}), \quad i = 1, \ldots, N,$$

so that

$$\mathrm{LSTM}_{n_h} : \mathbb{R}^{N \times H \times D} \to \mathbb{R}^{N \times n_h}.$$

C.2 Composite Structures

C.2.1 The Composite Convolutional Layer

The composite convolutional layer applies three blocks of

$$\text{Convolution} \;\to\; \text{LeakyReLU} \;\to\; \text{BatchNorm},$$

each with its own kernel size and stride. For some input in $\mathbb{R}^{N \times C \times H \times W}$ the composite convolutional layer applies a convolution filter of size $(1, k_w)$ and stride $(1, s_w)$. Observe this filter smooths the features in the space dimension. After applying a Leaky ReLU activation function and Batch Normalization, a time convolution filter of size $(4, 1)$ and stride $(1, 1)$ is applied. Then a Leaky ReLU and Batch Normalization are applied again. This last process is applied twice (see the composite convolution layer in figure **??** for a clear representation). Define the outer and inner layers to keep notation compact

$$\begin{aligned}
\mathrm{Outer}_{s_w}^{k_w}(T) &= \mathrm{BN}^{\phi,\varepsilon} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}_{1,s_w}^{32,1,k_w}(T),\\
\mathrm{Inner}^{(1)}(T) &= \mathrm{BN}^{\phi,\varepsilon} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{32,4,1}(T),\\
\mathrm{Inner}^{(2)}(T) &= \mathrm{BN}^{\phi,\varepsilon} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{32,4,1}(T).
\end{aligned}$$

Thus, the composite convolution layer with initial size $(1, k_w)$ and stride $(1, s_w)$ combines one outer layer and two inner layers as follows

$$\mathrm{CConv}_{s_w}^{k_w}(T) = \mathrm{Inner}^{(2)} \circ \mathrm{Inner}^{(1)} \circ \mathrm{Outer}_{s_w}^{k_w}(T).$$

Unless stated otherwise, no operations within the architecture share trainable parameters.

C.2.2 The Inception Layer

The inception layer is used to reduce the amount of computation needed to apply convolution filters in the deeper layers of a neural network **?**. It involves applying a set of $(1, 1)$ sized convolutional filters and max-pooling layers with padding are combined with an activation function and Batch Normalization. Then convolution operations of different size are applied and concatenated. Let us define the following layers

$$\begin{aligned}
\mathrm{IL}^{(1)}(T) &= \mathrm{BN}^{\phi,\varepsilon,(2)} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,3,1} \circ ZP^{1,0} \circ \\
&\qquad \mathrm{BN}^{\phi,\varepsilon,(1)} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,1,1}(T), \\
\mathrm{IL}^{(2)}(T) &= \mathrm{BN}^{\phi,\varepsilon,(2)} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,5,1} \circ ZP^{2,0} \circ \\
&\qquad \mathrm{BN}^{\phi,\varepsilon,(1)} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,1,1}(T), \\
\mathrm{IL}^{(3)}(T) &= \mathrm{BN}^{\phi,\varepsilon} \circ \mathrm{LeakyReLU}_{0.01} \circ \mathrm{Conv}^{64,3,1} \circ MP^{3,1}_{1,1} \circ ZP^{1,0}(T).
\end{aligned}$$

Once again, the superscripts in parentheses emphasize that the layers with similar notation do not share trainable parameters. The inception layer is the concatenation, along the second dimension, of these three tensors.

$$\mathrm{IL}(T) = \mathrm{Concat}^2(\mathrm{IL}^{(1)}(T), \mathrm{Concat}^2(\mathrm{IL}^{(2)}(T), \mathrm{IL}^{(3)}(T))).$$

C.3 Neural Network Architectures

C.3.1 The CNN-LSTM Architecture

The CNN-LSTM architecture of Kolm et al. **?**, which is depicted in figure **??**, is now formalized. As argued in Section **??**, there are multiple possibilities for the neural network output. The multihorizon approach explained this section is considered but the other possibilities can be constructed by modifying the outer layer. Let K be the output dimension. The CNN-LSTM neural network transformation is a map $\mathrm{CNNLSTM} : \mathbb{R}^{N\times 100\times 20} \to \mathbb{R}^{N\times K}$, where N is the training batch size, defined as

$$\mathrm{CNNLSTM}(\mathbf{X}) = \mathrm{FC}_K \circ \mathrm{LSTM}_{64} \circ \mathrm{Perm}^{3,2} \circ \mathrm{IL} \circ \mathrm{CConv}_1^{10} \circ \mathrm{CConv}_2^2(\mathbf{X}).$$

Here, $\mathbf{X}$ is a 3D tensor of whitened order flow matrices $\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i-99:i}$. That is,

$$\mathbf{X} = \left[\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_1-99:i_1}, \widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_2-99:i_2}, \ldots, \widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_N-99:i_N}\right].$$

Table **??** presents the intermediate dimensions of the layers with its respective trainable parameters assuming one horizon ($K = 1$) and a batch of size 32 ($N = 32$). The resulting model has $133,249$ parameters.

Layer	Notation	Input shape	Output shape	Trainable Parameters
Conv. Layer	CConv_2^2	$(32, 1, 100, 20)$	$(32, 32, 94, 10)$	8544
Conv. Layer	CConv_1^{10}	$(32, 32, 94, 10)$	$(32, 32, 88, 1)$	18720
Inception Layer	IL	$(32, 32, 88, 1)$	$(32, 194, 88, 1)$	39872
Permutation	$\mathrm{Perm}^{3,2}$	$(32, 192, 88, 1)$	$(32, 88, 192, 1)$	0
LSTM	LSTM_{64}	$(32, 82, 192)$	$(32, 64)$	66048
Fully Connected	FC_1	$(32, 64)$	$(32, 1)$	65

Table 3: CNN-LSTM architecture components.

During inference mode, the CNN-LSTM architecture will produce an output for each order flow matrix. Such output can be denoted $\mathrm{CNNLSTM}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i-99:i}\right)$. Moreover, the FC_K layer can be bypassed in order to obtain the penultimate layer representation which is effectively a feature map $\phi_{\mathbf{W}} : \mathbb{R}^{100\times 20} \to \mathbb{R}^{64}$ defined as

$$\phi_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i-99:i}\right) = \mathrm{LSTM}_{64} \circ \mathrm{Perm}^{3,2} \circ \mathrm{IL} \circ \mathrm{CConv}^{10}_{1} \circ \mathrm{CConv}^{2}_{2}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i-99:i}\right).$$

The subscript $\mathbf{W}$ is convenient to highlight the dependency on the parameters of the CNN-LSTM architecture which are updated iteratively.

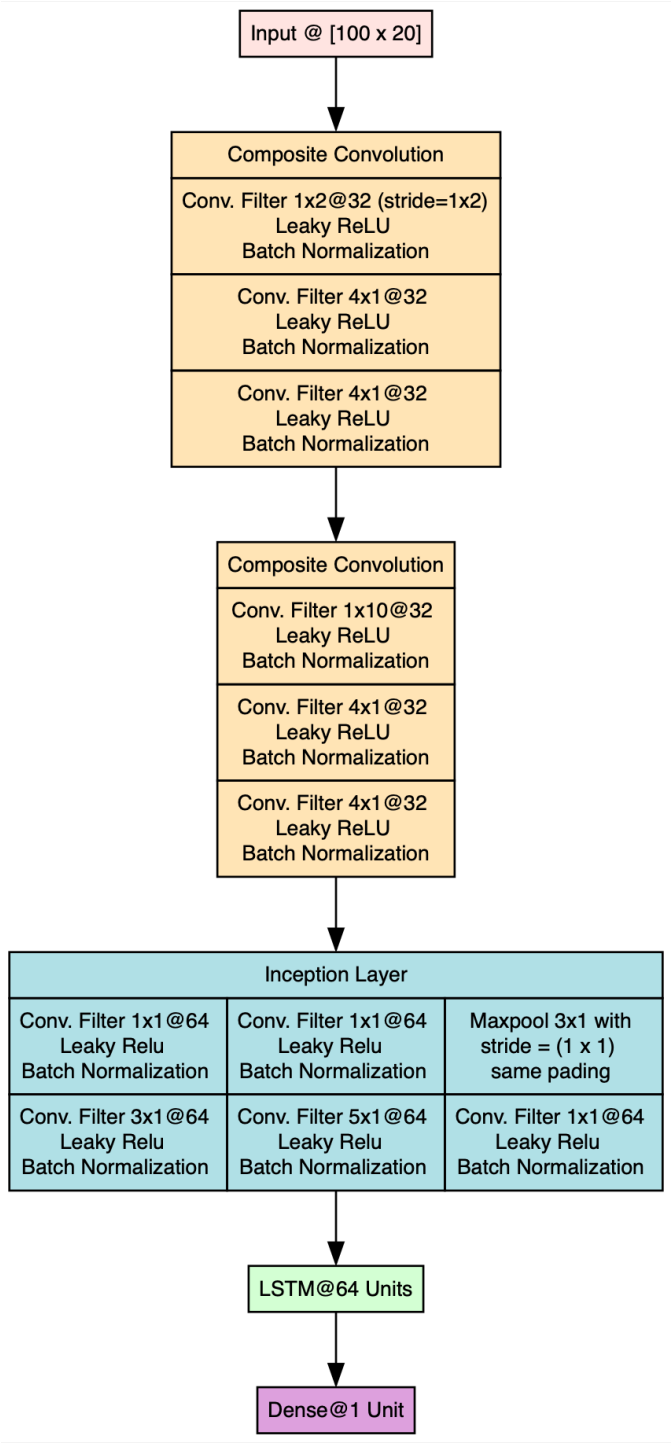


Figure 3: Illustration of the CNN-LSTM architecture.

C.3.2 The DeepLOB Architecture

Similarly, the DeepLOB architecture by Zhang et al. **?** adds an extra composite convolutional layer at the top of the CNN-LSTM architecture and takes raw prices and volumes as inputs. Formally DeepLOB :

$\mathbb{R}^{N\times 100\times 40} \to \mathbb{R}^{N\times K}$ with

$$\text{DeepLOB}(\mathbf{X}) = \text{FC}_K \circ \text{LSTM}_{64} \circ \text{Perm}^{3,2} \circ \text{IL} \circ \text{CConv}_1^{10} \circ \text{CConv}_2^{2,(2)} \circ \text{CConv}_2^{2,(1)}(\mathbf{X}).$$

Here, $\mathbf{X}$ is a 3D tensor of whitened limit order book price-volume information as defined in Section **??**:

$$\mathbf{X} = \left[\tilde{\mathbf{l}}^{\mathcal{P}^{(E)}}_{i_1-99:i_1}, \tilde{\mathbf{l}}^{\mathcal{P}^{(E)}}_{i_2-99:i_2}, \ldots, \tilde{\mathbf{l}}^{\mathcal{P}^{(E)}}_{i_N-99:i_N}\right]$$

Table **??** illustrates the components of this network assuming a scalar target variable ($K = 1$) and a batch of size 32 ($N = 32$). The resulting model comprises $143,777$ parameters.

Layer	Notation	Input shape	Output shape	Trainable Parameters
Conv. Layer	$\text{CConv}_2^{2,(1)}$	$(32, 1, 100, 40)$	$(32, 32, 94, 20)$	8544
Conv. Layer	$\text{CConv}_2^{2,(2)}$	$(32, 32, 94, 20)$	$(32, 32, 88, 10)$	10528
Conv. Layer	CConv_1^{10}	$(32, 32, 88, 10)$	$(32, 32, 82, 1)$	18720
Inception Layer	IL	$(32, 32, 82, 1)$	$(32, 194, 82, 1)$	39872
Permutation	$\text{Perm}^{3,2}$	$(32, 194, 82, 1)$	$(32, 82, 194, 1)$	0
LSTM	LSTM_{64}	$(32, 82, 194)$	$(32, 64)$	66048
Fully Connected	FC_1	$(32, 64)$	$(32, 1)$	65

Table 4: DeepLOB architecture components.

D Stationarity Testing

It is well known in forecasting theory that exogenous variables must be stationary to avoid spurious regressions **?**. Three main tests are adopted:

(A) Augmented Dickey–Fuller (ADF) Test. For any variable y (see Section **??**), the ADF test with local trend considers the following autoregressive model:

$$y_j = c + \big(1 - \phi_1 B - \phi_2 B^2 - \cdots - \phi_p B^p\big) y_j + \varepsilon_j,$$

which can be equivalently written as

$$\Delta y_j = \alpha + \gamma y_{j-1} + \sum_{k=1}^{p-1} \zeta_k \, \Delta y_{j-k} + \varepsilon_j, \tag{1}$$

where $\rho = \sum_{\ell=1}^{p} \phi_\ell$, $\zeta_k = -\sum_{\ell=k+1}^{p} \phi_\ell$, and B is the backshift operator. Note that $\gamma \le 0$, and $\gamma = 0$ corresponds to $\rho = 1$, indicating a unit root in $\phi(z) = 1 - \phi_1 z - \cdots - \phi_p z^p$. The ADF test compares

$$H_0^{\mathrm{ADF}} : \gamma = 0 \quad \text{versus} \quad H_1^{\mathrm{ADF}} : \gamma < 0.$$

Implementation details follow **?** and are available in the `fUnitRoots` package **?**. The test statistic is

$$Z_t^{\mathrm{ADF}} = \frac{\hat{\gamma}}{\mathrm{s.e.}(\hat{\gamma})},$$

where $\hat{\gamma}$ is estimated by ordinary least squares (OLS) on (**??**). Because Z_t^{ADF} follows a distribution derived from stochastic integrals rather than a Student's t-distribution, the critical values differ accordingly **?**.

A well-known drawback of the ADF test is its sensitivity to the choice of lag length p. In practice, this requires manual input or automated procedures. For simplicity, the current setup fixes $p = 10$ for all variables. Results from the ADF test are then compared with those of the PP test, which removes the lag-length requirement at the cost of reduced power.

(B) Phillips–Perron (PP) Test. The PP test uses a baseline model:

$$y_j = \rho \, y_{j-1} + u_j, \quad u_j = \sum_{k=0}^{\infty} \psi_k \, \varepsilon_{j-k} = \psi(B) \, \varepsilon_j,$$

where ε_j is an iid sequence of random variables with mean zero and variance σ^2. Assuming $\sum_{k=0}^{\infty} k \, \psi_k < \infty$, define $\gamma_k = \mathbb{E}[u_j \, u_{j-k}] = \sigma^2 \sum_{s=0}^{\infty} \psi_s \psi_{s+k}$ and $\lambda = \sigma \sum_{s=0}^{\infty} \psi_s = \sigma \, \psi(1)$. The test contrasts

$$H_0^{\mathrm{PP}} : \rho = 1 \quad \text{versus} \quad H_1^{\mathrm{PP}} : |\rho| < 1.$$

The statistic is

$$Z_\tau^{\mathrm{PP}} = \Big(\frac{\hat{\gamma}_0}{\hat{\lambda}^2}\Big)^{1/2} \frac{\hat{\rho} - 1}{\mathrm{s.e.}(\hat{\rho})} \; - \; \frac{1}{2}\Big(\hat{\lambda}^2 - \hat{\gamma}_0\Big)\Big(\frac{\hat{\lambda}^2}{\sum_{j=1}^{T} (y_j - \bar{y})^2}\Big)^{1/2},$$

where

$$\hat{\gamma}_k = \frac{1}{T} \sum_{j=k+1}^{T} \hat{u}_j \, \hat{u}_{j-k},$$

$$\hat{\lambda}^2 = \hat{\gamma}_0 + 2 \sum_{k=1}^{q} \Big(1 - \frac{k}{q+1}\Big) \hat{\gamma}_k,$$

$$q = \Big\lfloor 12 \Big(\frac{T}{100}\Big)^{1/4} \Big\rfloor.$$

Since Z_τ^{PP} depends on functionals of Brownian motion rather than standard distributions, critical values must be derived from the appropriate asymptotic theory. This test is implemented via the `urca` package **?**.

(C) KPSS Test. For completeness, the Kwiatkowski–Phillips–Schmidt–Shin (KPSS) test **?** is included in a confirmatory role. Unlike the ADF and PP tests, the KPSS null hypothesis assumes stationarity. The model is

$$y_j = r_j + u_j, \quad r_j = r_{j-1} + e_j,$$

where e_j is an iid sequence with zero mean and variance σ_e^2, and u_j satisfies properties similar to those in the PP framework. That is $u_j = \psi(B)\varepsilon_j$, for some $\varepsilon \overset{iid}{\sim} (0, \sigma^2)$, and $\sum_{k=0}^{\infty} k\psi_k < \infty$. The hypotheses are

$$H_0^{KPSS} : \sigma_e^2 = 0 \quad \text{versus} \quad H_1^{KPSS} : \sigma_e^2 > 0.$$

The test statistic is

$$\hat{\eta}_{\mu}^{\text{KPSS}} = \sum_{k=1}^{T} \frac{S_k^2}{\hat{\lambda}^2},$$

where $S_k = \sum_{j=1}^{k} \hat{u}_j$ represents the partial sum of residuals ($\hat{u}_j = y_j - \bar{y}$), and $\hat{\lambda}^2$ follows the same form as in the PP test. As in the previous tests, critical values derive from nonstandard limiting distributions. The `urca` package used for computing the rejection region of the this test.

E Case Study: Model Subspace Selection

This Monte Carlo study compares adaptive LASSO (adaLASSO) (**??**) with OLS-Bonferroni selection for exact support recovery and examines how quickly the failure probability decays to zero. The study is organized as follows. Section **??** reviews the Medeiros & Mendes framework describing the DGP and assumptions presented in **?**. Section **??** describes the particular DGP used for the simulations and the dynamics of design space growth. Section **??** analyzes the simulated data to quantify the effect of multicollinearity and initial weights selection. Section **??** draws conclusions and recommendations and Section **??** provides precise instructions for the replication of the present study.

E.1 Background

This section discusses the key regularity conditions and results in **?** and provides some intuitive interpretation to these ideas.

First, consider the data generating process (DGP)

$$y_t = \alpha_0 + \boldsymbol{\theta}^T \mathbf{x}_t + u_t. \tag{2}$$

Where $\{\mathbf{x}_t\}_{t\in\mathbb{Z}}$ is an n_T-dimensional covariate process, which possibly contains lags of the endogenous process, and $\{u_t\}_{t\in\mathbb{Z}}$ is a martingale difference sequence. Denote by T the sample size of observations of the process $\{y_t\}_{t\in\mathbb{Z}}$, n_T the total number of covariates, and s_T the number of relevant covariates. For simplicity assume that the first s_T elements of $\mathbf{x}_t$ correspond to the relevant variables, which can effectively be denoted as $\mathbf{x}_t = [\mathbf{x}_t(1), \mathbf{x}_t(2)]$. Also write $\boldsymbol{\theta}^T = [\boldsymbol{\theta}^T(1), \boldsymbol{\theta}^T(2)]$ and let $\boldsymbol{\theta}_0$ denote the true parameter which effectively implies that $\boldsymbol{\theta}_0^T = [\boldsymbol{\theta}_0^T(1), \mathbf{0}^T]$. Assuming, without loss of generality, that $\alpha_0 = 0$, The adaLASSO technique consists in minimizing the objective function given by

$$f(\boldsymbol{\theta}) = \left\|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\right\|_2^2 \; + \; \lambda_T \sum_k w_k \left|\theta_k\right|.$$

Where $w_k = \left|\theta_{I,k}\right|^{-\tau}$ are computed from $\tau > 0$ and $\boldsymbol{\theta}_I$ an initial parameter estimate. The initial parameter estimate could be given, among others, by OLS, LASSO or elastic-net. The choice of the algorithm for the initial parameters is left out of the scope of this simulation study. Medeiros and Mendes group their assumptions into four categories: Data Generating Process (DGP), Design Space (DESIGN), Initial Weights (WEIGHTS), and Regularization Parameter (REG). These assumptions are restated below for self-completeness.

(DGP1) Define $\mathbf{Z}_t = (\mathbf{x}_t^T,\, u_t)^T$. $\{\mathbf{Z}_t\}_{t\in\mathbb{Z}}$ is zero–mean and weakly stationary.

(DGP2) $\mathbb{E}\left\{u_t \mid \mathcal{F}_t\right\} = 0$, where $\mathcal{F}_t = \sigma(\mathbf{x}_t, \mathbf{Z}_{t-1}, \mathbf{Z}_{t-2}, \dots)$.

(DGP3)

$$\max_{1\le i\le n_T} \left|\tfrac{1}{T}\sum_{t=1}^{T}(x_{it}^2 - E[x_{it}^2])\right| \xrightarrow{\mathbb{P}} 0.$$

(DGP4) For some $m \ge 2$, there exists $c_m < \infty$ such that

$$\mathbb{E}\left|x_{it}\, u_t\right|^m \le c_m \quad \forall t \in \mathbb{Z}, i \in 1 : n_T.$$

(DESIGN1) θ_0 is an open subset $\boldsymbol{\Theta}_{n_T} \subseteq \mathbb{R}^{n_T}$ which contains $\mathbf{0} \in \mathbb{R}^{n_T}$.

(DESIGN2) There exists $\theta_{\min} > 0$ such that $\min_{i\le s_T} |\theta_{0,i}| > \theta_{\min}$.

(DESIGN3) Denote $\hat{\boldsymbol{\Omega}}_{11} := \frac{1}{T}\mathbf{X}^T(1)\mathbf{X}(1)$ and $\boldsymbol{\Omega}_{11} := \mathbb{E}\left\{\mathbf{x}_t(1)\mathbf{x}_t^T(1)\right\}$

(a) There exists a constant $\phi_{\min} \in (0,1)$ such that

$$\inf_{\|\boldsymbol{\alpha}\|=1} \boldsymbol{\alpha}^T \boldsymbol{\Omega}_{11} \boldsymbol{\alpha} > 2\phi_{\min}.$$

Note that this is equivalent to the smallest eigenvalue of $\boldsymbol{\Omega}_{11}$ being greater than $2\phi_{\min}$.

(b) $\|\boldsymbol{\Omega}_{11} - \hat{\boldsymbol{\Omega}}_{11}\|_\infty = \max_{ij} \left|\boldsymbol{\Omega}_{11} - \hat{\boldsymbol{\Omega}}_{11}\right|_{ij} \le \frac{\phi_{\min}}{s_T}$

(WEIGHTS1) The weights on inactive endogenous variables diverge fast enough: for some $\xi \in (0,1)$ and $c_w > 0$,

$$\min_{i \in s_T+1:n_T} T^{-\xi/2}\, w_i \;>\; c_w \sqrt{\frac{s_T}{\phi_{\min}}}.$$

with probability converging to one

(WEIGHTS2) The weights on the active set are uniformly bounded:

$$\sum_{i=1}^{s_T} w_i^2 < s\, w_{\max,T}^2, \quad w_{\max,T} < T^{\xi/2},$$

with probability converging to one.

(REG) Let $\lambda_T > 0$ be the Adaptive-LASSO penalty parameter. It is required that

$$\frac{n_T^{1/m} T^{(1-\xi)/2}}{\lambda_T} \xrightarrow[T\to\infty]{} 0 \quad \text{and} \quad \frac{\lambda_T}{\sqrt{T}} \frac{s_T^{1/2}}{w_{\max,T}\, \phi_{\min}} \xrightarrow[T\to\infty]{} 0.$$

Further examination of these assumptions reveal interesting patterns. For analysis convenience, define the following dynamics

$$\begin{aligned} n_T &= \lceil c_\delta\, T^\delta \rceil, \\ s_T &= \lceil c_\gamma\, T^\gamma \rceil, \\ \lambda_T &= c_\alpha n_T^{1/m}\, T^{\frac{1-\xi}{2}+\frac{\alpha}{m}}, \\ w_{S,T}^2 &= c_{\beta_S}\, T^{\beta_S}, \quad w_{S^c,T}^2 = c_{\beta_{S^c}}\, T^{\beta_{S^c}}. \end{aligned}$$

Here, $w_{S,T}$ and $w_{S^c,T}$ model the growth order of the active and inactive weights respectively. Basic calculus shows that the assumptions (WEIGHTS1), (WEIGHTS2), and (REG) above can be rewritten as

$$\begin{aligned} \beta_{S^c} - \xi > \gamma, & \qquad \text{(WEIGHTS1)} \\ \beta_S < \xi, & \qquad \text{(WEIGHTS2)} \\ \alpha > 0, \quad \frac{2}{m}\big(\alpha + \delta\big) < \xi - \gamma - \beta_S. & \qquad \text{(REG)} \end{aligned}$$

These inequalities reveal interesting relationships under the light of the following results.

Theorem 1 (Theorem 1 of **?**)**.** *Under Assumptions* DGP, DESIGN, WEIGHTS *and* REG, *suppose moreover that*

$$\theta_{\min} > \frac{\lambda_T\, s_T^{1/2}}{T^{1-\xi/2}\, \phi_{\min}}.$$

Then the adaptive LASSO estimator $\hat{\theta}$ satisfies

$$\mathbb{P}\big(\operatorname{sign}(\hat{\theta}) = \operatorname{sign}(\theta_0)\big) \xrightarrow[T\to\infty]{} 1$$

In the theorem above, the sign function is defined as $\operatorname{sign}(x) = I_{\{x>0\}} - I_{\{x<0\}}$ which implies that $\operatorname{sign}(0) = 0$.

Theorem 2 (Theorem 2 of **?**)**.** *Let $\hat{\boldsymbol{\theta}}_{ols}(1)$ denote the OLS estimator of the true active coefficients $\boldsymbol{\theta}_0(1)$. Under the assumptions of Theorem* **??**, *it holds that for any s_T-vector $\boldsymbol{\alpha}$ with $\|\boldsymbol{\alpha}\|_2 = 1$,*

$$\sqrt{T}\, \boldsymbol{\alpha}^T \left(\hat{\boldsymbol{\theta}}(1) - \boldsymbol{\theta}_0(1)\right) = \sqrt{T}\, \boldsymbol{\alpha}^T \left(\hat{\boldsymbol{\theta}}_{ols}(1) - \boldsymbol{\theta}_0(1)\right) \;+\; o_p(1).$$

Theorem 2—often called the *oracle property*—goes one step further: not only does the adaptive LASSO select the correct support with probability tending to one, but its active-set estimates are asymptotically as efficient as if we knew the true model in advance. Moreover, the proof of Theorem 1 shows that, for some $c > 0$ and for a sufficiently large T,

$$\mathbb{P}\big(\text{sign}(\hat{\theta}) \neq \text{sign}(\theta_0)\big) \;\leq\; \frac{c}{T^{\alpha}}, \tag{3}$$

so the probability of selection failure decays at rate $T^{-\alpha}$. In practice this means that larger α yields faster convergence—yet α itself is bounded by the REG assumption's *window*. Note also that heavier-tailed errors (smaller m) tighten this window, as do steeper growth in the number of covariates, denser true parameter vectors θ_0, or more aggressive penalization on the active set.

E.2 Simulation Design

Simulated data is generated from the DGP

$$y_t = \phi y_{t-1} \;+\; \sum_{j=1}^{n_T} \beta_j \, x_{jt} \;+\; u_t,$$

where,

$$\begin{cases} \mathbf{x}_t = \phi_X \, \mathbf{x}_{t-1} \;+\; \mathbf{L}\,\mathbf{v}_t, & v_{jt} \overset{iid}{\sim} t_{m+1}, \\ u_t = \sigma_t \eta_t & \eta_t \overset{iid}{\sim} t_{m+1}, \\ \sigma_t^2 = \omega + \alpha_\star u_{t-1}^2 + \beta_\star \sigma_{t-1}^2. \end{cases}$$

Where ϕ_X is a scalar and $\mathbf{L}$ is chosen to be a lower triangular matrix such that $\mathbf{R} := \mathbf{L}\mathbf{L}^{\mathbf{T}}$, where $\mathbf{R}$ is the desired correlation matrix. Take $\beta_j = 1$ if $j \leq s_T$ and zero otherwise. Take $\phi = 0.8$, $\phi_X = 0.5$, $\omega = 0.1$, $\alpha_\star = 0.1$, $\beta_\star = 0.8$, and $m = 8$. Construct $\mathbf{R}$ starting from the identity matrix and replace its entries $(1,2)$ and $(2,1)$ with ρ (i.e. $\mathbf{R}_{1,2} = \mathbf{R}_{2,1} = \rho$), where $\rho \in \{0, 0.25, 0.5, 0.75, 0.99\}$. The parameters governing the dynamics of the number of candidate variables and penalizing weights are given below.

c_α	α	c_{β_S}	β_S	$c_{\beta_{S^c}}$	β_{S^c}	c_γ	γ	c_δ	δ	ξ
1	0.25	1	0.125	1	1.5	2	0.25	1	0.5	0.75

Table 5: Simulation parameters.

Partitioning the interval $[64, 1024]$ into 15 roughly equally-spaced values of T produces the simulation grid shown in Table **??**. Each (T, n_T, s_T) configuration is then replicated $N = 100$ times for each of the specified correlation levels. For each of the samples, adaLASSO is fit assuming equal weights for the active set ($w_{S,T}$) and weights equal to $w_{S^c,T}$ for the inactive set. This helps to control the dynamics of the penalizing weights which in practice emerge from τ and the initial estimate selection algorithm. For simplicity, the weight corresponding to the lagged value of y_t is left close to zero and only one lag is included in $\mathbf{X}$. OLS is then fit and the variance of $\hat{\tilde{\boldsymbol{\beta}}}$ is estimated via the homoskedastic formula (as a benchmark) and the heteroskedasticity-consistent estimator (HCE). Specifically

$$\hat{V}_{\text{hom}} = \hat{\sigma}^2 (\tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{X}})^{-1}; \quad \hat{\sigma}^2 = \frac{1}{T - n_T - 1} \sum_{t=1}^{T} \hat{u}_t^2,$$

$$\hat{V}_{\text{het}} = (\tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{X}})^{-1} \left(\sum_{t=1}^{T} \tilde{\mathbf{x}}_t \tilde{\mathbf{x}}_t^T \hat{u}_t^2 \right) (\tilde{\boldsymbol{X}}^T \tilde{\boldsymbol{X}})^{-1}.$$

Where in the above the *tilded* quantities denote the corresponding augmented quantity to account for the inclusion of one lag of the endogenous variable. Statistical significance is assessed throught the t test with Bonferroni correction. That is, a variable will be selected following each of the OLS approaches iff

$$\frac{\hat{\tilde{\beta}}_i}{V_{ii}^{1/2}} > t_{T-s_T-1,1-\frac{0.025}{(n_T+1)}}.$$

Following the execution of these three variable selection algorithms. The proportion of correctly selected models is calculated for each of the different $T\rho$-combinations.

T	n_T	s_T
64	8	6
133	12	7
202	15	8
270	17	9
339	19	9
407	21	9
476	22	10
544	24	10
613	25	10
682	27	11
750	28	11
819	29	11
887	30	11
956	31	12
1024	32	12

Table 6: Simulation grid: sample size T and corresponding $n_T = \lceil c_\delta T^\delta \rceil$, $s_T = \lceil c_\gamma T^\gamma \rceil$.

E.3 Results and Discussion

E.3.1 Multicollinearity Effect

Figure **??** (see section **??**) displays the exact recovery rate (fraction of simulations where $\text{sign}(\hat{\boldsymbol{\theta}}) = \text{sign}(\boldsymbol{\theta})$) as a function of T, for different correlation levels ρ. Several clear patterns emerge. First, as expected, increasing the sample size improves support recovery for both methods in all scenarios – the curves generally rise toward 1.0 as T grows. However, the adaLASSO (red curves) achieves much higher recovery rates at moderate sample sizes than OLS (blue/green curves) in the presence of strong multicollinearity. For low correlation ($\rho = 0$ or 0.25), the gap between methods is small: OLS with Bonferroni performs nearly as well as adaLASSO when predictors are independent or only mildly correlated. In fact, for $\rho = 0$, both methods eventually reach 100% recovery with sufficiently large T (hundreds of observations), and their trajectories are quite close. This is because when predictors are uncorrelated, the OLS t-tests remain informative (each coefficient can be estimated independently with little variance inflation), so the Bonferroni penalty for multiple testing is the main hurdle – which large T can overcome by reducing standard error. AdaLASSO also works well in this regime, effectively similar to the ordinary LASSO which is known to be consistent under mutual independence of features.

When correlation increases to moderate levels ($\rho = 0.5$ shown in Figure **??**, top-right panel, and $\rho = 0.75$ in bottom-left), the performance of OLS selection deteriorates dramatically relative to adaLASSO. For $\rho = 0.5$, the OLS method (blue/green lines) requires a much larger T to approach high recovery probability – there is a noticeable rightward shift of the OLS curve compared to the adaLASSO curve. The adaLASSO achieves near-perfect support recovery at $T = 339$ observations when $\rho = 0.5$, whereas OLS-Bonferroni needs roughly double that sample size to reach comparable accuracy. The gap widens further at $\rho = 0.75$: here the OLS approach almost never succeeds for small or moderate samples (recovery probability essentially 0 for $T < 300$). Even at $T = 1024$, OLS is only recovering the true model in about 50–60% of trials, whereas adaLASSO already exceeds 90% success by that point. the adaLASSO appears more robust to multicollinearity when T is large enough, eventually achieving near-certain recovery even at $\rho = 0.75$ in the simulations (red curve tending to 1.0 as T grows). OLS, however, remains very sensitive: even at $T = 1024$, the OLS method never reaches 100% recovery for $\rho = 0.75$.

The most extreme case is $\rho = 0.99$ (Figure **??**, bottom-middle), where predictors are almost perfectly correlated. This scenario is akin to having duplicate or nearly-duplicate variables in the dataset –

a problem for OLS inference. Accordingly, the OLS selection method essentially fails entirely: the blue/green curves at $\rho = 0.99$ sit near zero across all sample sizes tested. The adaLASSO (red curve) also finds this scenario challenging (its recovery probability grows only slowly with T and does not reach 1 even at $T = 1024$), but it clearly outperforms OLS: by $T = 1024$ observations, adaLASSO achieves about 70% exact recovery, whereas OLS is essentially at 0%. This sensitivity to multicollinearity of the adaLASSO approach connects with the assumption DESIGN3 as $\phi_{\min}$ decreases with ρ. The simulation confirms that multicollinearity breaks down the performance of naive OLS variable selection, while adaLASSO is far more resilient in discovering the correct predictors (especially as T becomes large).

To connect with the theoretical result from **?** (see **??**), the rate at which the failure probability decreases with T is examined. Figure **??** plots the Monte Carlo estimated failure rates on logarithmic scales. If the failure probability follows a power-law $T^{-\alpha}$, an approximately straight line with slope $-\alpha$ in the log–log representation would be observed. Indeed, for the adaLASSO (red) a roughly linear downward trend in all panels is seen - except when the curve saturates -, consistent with a polynomial decay in T. At every correlation level, the estimated failure rate (red) declines faster than the theoretical bound line (dashed), indicating a steeper power-law exponent. Thus, the adaLASSO simulation results are consistent with the theoretical guarantee of polynomial-decay consistency: even under dependence and heavy-tailed noise, it is observed that $\mathbb{P}\big(\text{sign}(\hat{\theta}) \neq \text{sign}(\theta_0)\big)$ trends down at least as fast as $cT^{-\alpha}$ for some $\alpha > 0$.

The OLS-based methods (blue/green lines in figure **??**) tell a similar story when the multicollinearity level is below or at 50%. However, the failure probability rate of decrease exhibits a slower convergence when the multicollinearity level is at or above 75%. This reflects the fact that OLS with Bonferroni can eventually consistently select the true model when predictors have low to moderate correlation. For higher correlations ($\rho \geq 0.75$), the blue/green points in Figure **??** are almost horizontal or very mildly declining on the log–log scale. This underscores the superior performance of adaLASSO under high multicollinearity regimes when the DGP, DESIGN, WEIGHTS, and REG conditions hold.

E.3.2 Effect of Initial Weights

It might be suggested that adaLASSO may enjoy an unfair edge under the simulation configuration presented, since the true parameter values are "encoded" in the evolution of the penalty weights w_i. To address this concern, a targeted sensitivity experiment was conducted: setting the correlation parameter $\rho = 0$ and raising the inactive-set weights to $\beta_{S^c} = 1.15$, while holding all other hyper-parameters fixed. The results (Figures **??** and **??**) show that, under these conditions, adaptive-LASSO requires substantially larger sample sizes before its exact-recovery probability saturates. Crucially, however, once the sample size exceeds a threshold, the failure probability decays very rapidly, in line with the oracle-type behavior predicted by theory.

These findings make clear that careful tuning of the initial-weight construction is important when T is limited or the signal in the inactive set is relatively strong.

E.4 Conclusions and Recommendations

In this Monte Carlo case study the adaLASSO and Bonferroni-corrected t-tests were compared, focusing on exact support recovery and the empirical rate at which the failure probability decays to zero. The main findings are:

- **Exact recovery vs. multicollinearity.** When predictors are uncorrelated or only mildly correlated ($\rho \leq 0.25$), both methods eventually recover the true support with high probability as T grows. However, under moderate to strong multicollinearity ($\rho \geq 0.5$), OLS-Bonferroni requires much larger T and in extreme cases ($\rho = 0.99$) essentially fails, whereas adaLASSO still achieves a decent recovery rate for moderately large samples.

- **Polynomial decay of failure rate.** On a log–log scale, the adaLASSO's failure probability exhibits an approximately linear decline of slope $-\alpha$, consistent with the theoretical bound

$$\Pr\big(\text{sign}(\hat{\theta}) \neq \text{sign}(\theta_0)\big) \;\lesssim\; cT^{-\alpha}.$$

  In contrast, OLS-Bonferroni only attains an adaLASSO-comparable decay when ρ is small; its curve flattens out for higher correlations.

- **Sensitivity to initial-weight tuning.** By increasing the inactive-set exponent β_{S^c}, it was shown that adaLASSO may require larger T before its recovery probability "saturates." Yet once past a moderate threshold, its convergence is very rapid, demonstrating the oracle-property in practice. This underscores the importance of the selection of an initial weights selection method that discriminates inactive variables.

Practical recommendations. Based on these results, practitioners should:

- Employ adaLASSO in high-dimensional, time-series settings where multicollinearity is present, since it substantially outperforms OLS-based selection under dependence.
- Carefully tune the hyperparameters of the initial weights selection.
- Use OLS-Bonferroni as a simple baseline when predictors are mildly correlated.
- Validate selection performance before deploying the method in empirical applications.

Overall, the present study confirms that adaLASSO not only achieves model selection consistency and the oracle property under the Medeiros & Mendes framework, but also delivers robust performance in finite samples—especially when multicollinearity and heteroskedastic, heavy-tailed errors are present.

E.5 Reproducibility

All the simulation results presented here can be reproduced using the code available in the GitHub repository `judmejiabe/case-study-adaLASSO`. The repository contains detailed scripts for generating the data, running the adaptive LASSO and OLS selection procedures, and producing the figures.

F Events Matching Exclusion Criteria on the INTC MBO Dataset

This section lists the dates excluded from the analysis as a consequence of meeting any of the excluding criteria highlighted in Section **??**. The exclusion criteria is reproduced below for the reader's convenience.

- Corporate actions: Ex-dividend dates, stock splits or reverse splits, share buybacks and rights issues, spin-offs, mergers, or acquisitions
- Earnings and company-specific news: Quarterly earnings announcements and guidance revisions, joint ventures, large partnerships or asset sales, regulatory actions or class-action lawsuits, product launches or major technological disclosures
- Derivative market events: Monthly options expirations (OPEX), quarterly triple witching weeks (stock index futures, index options, and single-stock options expiry), index futures rollovers (e.g., NASDAQ-100, S&P 500), index and ETF quarterly rebalancing
- Scheduled announcements: U.S. Employment Situation (non-farm payrolls), Consumer Price Index (CPI), Producer Price Index (PPI), PCE releases, FOMC interest rate decisions and press conferences, other major macroeconomic indicators (e.g., GDP, JOLTS)
- Exchange halts: Limit-Up/Limit-Down (LULD) volatility pauses, market-wide circuit breakers, exchange-initiated suspensions (technical or regulatory).

F.1 Corporate Actions

- May 6, 2024. Event Type: Ex-dividend date. Description: Ex-dividend date for quarterly cash dividend of $0.125 per share. Source: `https://stocklytics.com/stocks/intc/dividends`

F.2 Earnings and Company-Specific News

- April 25, 2024 (after market close, impacting May 1). Event Type: Earnings release. Description: Intel reported Q1 2024 earnings, the first quarter under its internal foundry reporting model; the Foundry segment revenue was $4.4 billion (down 10% year-over-year). Source: `https://www.intc.com/news-events/press-releases/detail/1635/intel-reports-first-quarter-2024-financial-results`
- May 8, 2024. Event Type: Regulatory action (export license revocation). Description: The U.S. Commerce Department revoked certain export licenses for shipments to Huawei, affecting Intel's ability to supply that customer. Source: `https://www.marketwatch.com/story/intels-stock-drops-after-u-s-revokes-licenses-for-exports-to-china-customer-f057da21`
- June 4, 2024. Event Type: Joint venture. Description: Intel and Apollo announced a definitive agreement under which Apollo acquired a 49% equity stake in a joint venture related to Intel's Fab 34 facility in Leixlip, Ireland (Intel retains operational control). Source: `https://newsroom.intel.com/corporate/co-invest-program-news-2024`
- June 17, 2024. Event Type: Class action lawsuit. Description: Bronstein, Gewirtz & Grossman announced a securities-fraud class action against Intel regarding statements about the internal foundry model and foundry losses (class period Jan 25–Apr 25, 2024). Source: `https://www.globenewswire.com/de/news-release/2024/05/13/2880132/0/en/INTC-INVESTOR-ALERT-Bronstein-Gewirtz-Grossman-LLC-Announces-that-Intel-Corporation-Investors-with-Substantial-Losses-Have-Opportunity-to-Lead-Class-Action-Lawsuit.html`
- June 21, 2024. Event Type: Class action lawsuit notice. Description: Levi & Korsinsky announced a class action filing and lead-plaintiff deadline of July 2, 2024, reiterating allegations concerning Intel's internal foundry disclosures. Source: `https://www.prnewswire.com/news-releases/class-action-filed-against-intel-corporation-intc---july-2-2024-deadline-to-join--contact-levi--korsinsky-302178578.html`

F.3 Derivative Market Events

- May 17, 2024. Event Type: Options expiration (OPEX). Description: Monthly U.S. equity and ETF options expiration. Source: https://www.macroption.com/options-expiration-calendar/
- June 17-21, 2024. Event Type: Triple witching week. Description: Simultaneous expiration of stock index futures, stock index options, and single-stock options, coinciding with June OPEX. Source: https://www.bankrate.com/investing/triple-witching/

F.4 Scheduled Announcements

- May 3, 2024. Event Type: U.S. Employment Situation (April NFP). Description: BLS reported non-farm payrolls at 175k (below consensus), released at 8:30a ET. Source: https://www.bls.gov/news.release/archives/empsit_05032024.htm
- May 15, 2024. Event Type: CPI release (April). Description: Consumer Price Index data released at 8:30a ET. Source: https://www.bls.gov/news.release/archives/cpi_05152024.htm
- June 4, 2024. Event Type: JOLTS (Job Openings). Description: The Bureau of Labor Statistics released the April Job Openings and Labor Turnover Survey at 10:00 a.m. ET. Source: https://www.bls.gov/news.release/archives/jolts_06042024.htm
- June 7, 2024. Event Type: U.S. Employment Situation (May NFP). Description: Non-farm payrolls release at 8:30a ET. Source: https://www.bls.gov/news.release/archives/empsit_06072024.htm
- June 12, 2024. Event Type: CPI release (May). Description: Consumer Price Index release at 8:30a ET. Source: https://www.bls.gov/news.release/archives/cpi_06122024.htm
- June 12, 2024. Event Type: FOMC decision. Description: Federal Reserve announced policy decision and released updated Summary of Economic Projections at 2:00p ET, followed by press conference. Source: https://www.federalreserve.gov/monetarypolicy/fomccalendars.htm
- June 13, 2024. Event Type: PPI release (May). Description: Producer Price Index release at 8:30a ET. Source: https://www.bls.gov/news.release/archives/ppi_06132024.htm
- June 14, 2024. Event Type: Import/Export Prices (May). Description: BLS Import and Export Price Indexes released at 8:30a ET. Source: https://www.bls.gov/news.release/archives/ximpim_06142024.htm

F.5 Exchange Halts

- None. No Nasdaq trading halts were recorded for INTC between May 1 and June 30, 2024. Source: Nasdaq Trader Trading Halts database https://www.nasdaqtrader.com/Trader.aspx?id=TradeHalts

G Complimentary Tables and Figures

Category	Parameter	Value
General	Ticker	SIM (Simulated LOB)
	Δ^B	$5'000{,}000.0$ uot
	Δ^A	100,000.0 uot
	δ	100.0 uot
	Seed	19991810
	Session start	0.0 uot
	Session finish	$10'000{,}000.0$ uot
	Target Variable	r^w
Backbone	Training Epochs	10
	Batch size	32
	Validation proportion	0.1
	Buffer proportion	0.05
	Windsorization quantile	0.005
	ADF rejection threshold	0.8
	KPSS verification threshold	0.2
	NA threshold	0.1
	Learning rate	0.0001
	Correlation threshold	0.95
Interface	α	$\{0.0, 0.1, 0.25, 0.5, 0.75, 0.9, 1.0\}$
	λ_1	logspace$(-4, 1, 25)$
	τ	$\{0.25, 0.5, 1.0, 1.5, 2.0\}$
	λ_2	logspace$(0, 1, 25)$
	Validation proportion	0.2
	Selection threshold	0.17 ($L = 50$, $\alpha = 0.00005$)
	Tolerance threshold	10^{-6}
	OLS significance level	0.05
	Surrogate model lags	5
Adapter	ARMA order	(1,1)
	GARCH order	(1,1)
	Error distribution	Skewed t

Table 7: Base configuration used in synthetic data experiments.

Category	Parameter	Value
General	Ticker	INTC
	Δ^B	7,200.0 sec
	Δ^A	600.0 sec
	δ	2.0 sec
	Seed	One per date pair
	Session start	9:45 AM EST
	Session finish	3:45 PM EST
	Target Variable	r^w
Backbone	Training Epochs	10
	Batch size	32
	Validation proportion	0.1
	Buffer proportion	0.05
	Windsorization quantile	0.005
	ADF rejection threshold	0.8

Table 8 (continued)

Category	Parameter	Value
	KPSS verification threshold	0.2
	NA threshold	0.2
	Learning rate	0.0001
	Correlation threshold	0.95
	α	{0.0, 0.1, 0.25, 0.5, 0.75, 0.9, 1.0}
	λ_1	logspace$(-4, 1, 25)$
	τ	{0.25, 0.5, 1.0, 1.5, 2.0}
	λ_2	logspace$(0, 1, 25)$
Interface	Validation proportion	0.2
	Selection threshold	0.29 ($L = 12$, $\alpha = 0.00005$)
	Tolerance threshold	10^{-6}
	OLS significance level	0.05
	Surrogate model lags	5
	ARMA order	(0,1)
Adapter	**EGARCH** order	(1,1)
	Error distribution	Negative Inverse Gaussian
	Update Rule	Exponential Smoothing

Table 8: Base configuration used in the experiments with market data.

Table 9: RelMAE-based overall rankings and average RelMAE values for each TTSTL configuration across the backbone bars in the INTC MBO dataset. DP denotes the date pair, UR the update rule, CC the covariate configuration, Rank BBX the overall ranking within backbone bar X, and RelMAE BBX the corresponding average forecasting error.

DP	UR	CC	Rank BB0	RelMAE BB0	Rank BB1	RelMAE BB1	Rank BB2	RelMAE BB2
1	ES010	01	3.0	99.98%	3.0	100.63%	3.0	99.17%
1	ES010	02	3.0	99.98%	3.0	100.63%	3.0	99.17%
1	ES010	03	3.0	99.98%	3.0	100.63%	3.0	99.17%
1	ES010	04	3.0	99.98%	3.0	100.63%	3.0	99.17%
1	ES010	05	6.0	100.02%	6.0	100.75%	6.0	99.20%
1	ES010	06	3.0	99.98%	3.0	100.63%	3.0	99.17%
2	ES010	01	3.0	99.84%	3.0	100.56%	4.0	100.39%
2	ES010	02	3.0	99.84%	3.0	100.56%	4.0	100.39%
2	ES010	03	3.0	99.84%	3.0	100.56%	4.0	100.39%
2	ES010	04	3.0	99.84%	3.0	100.56%	4.0	100.39%
2	ES010	05	6.0	100.43%	6.0	100.75%	1.0	100.32%
2	ES010	06	3.0	99.84%	3.0	100.56%	4.0	100.39%
3	ES010	01	3.0	99.85%	3.0	100.17%	2.5	100.53%
3	ES010	02	3.0	99.85%	3.0	100.17%	5.5	100.59%
3	ES010	03	3.0	99.85%	3.0	100.17%	2.5	100.53%
3	ES010	04	3.0	99.85%	3.0	100.17%	2.5	100.53%
3	ES010	05	6.0	99.91%	6.0	100.32%	5.5	100.59%
3	ES010	06	3.0	99.85%	3.0	100.17%	2.5	100.53%
4	ES010	01	3.0	99.64%	4.5	102.53%	4.0	101.48%
4	ES010	02	3.0	99.64%	1.5	101.04%	4.0	101.48%
4	ES010	03	3.0	99.64%	4.5	102.53%	4.0	101.48%
4	ES010	04	3.0	99.64%	4.5	102.53%	4.0	101.48%
4	ES010	05	6.0	100.45%	1.5	101.04%	1.0	101.38%
4	ES010	06	3.0	99.64%	4.5	102.53%	4.0	101.48%
5	ES010	01	3.0	99.93%	4.5	99.80%	4.0	99.77%
5	ES010	02	3.0	99.93%	1.5	99.76%	4.0	99.77%

Continued on next page

Table 9 – continued from previous page

DP	UR	CC	Rank BB0	RelMAE BB1	Rank BB1	RelMAE BB2	Rank BB3	RelMAE BB2
5	ES010	03	3.0	99.93%	4.5	99.80%	4.0	99.77%
5	ES010	04	3.0	99.93%	4.5	99.80%	4.0	99.77%
5	ES010	05	6.0	100.06%	1.5	99.76%	1.0	99.77%
5	ES010	06	3.0	99.93%	4.5	99.80%	4.0	99.77%
6	ES010	01	4.0	100.40%	3.5	102.23%	2.5	100.14%
6	ES010	02	4.0	100.40%	5.5	131.02%	5.5	127.17%
6	ES010	03	4.0	100.40%	1.5	100.67%	2.5	100.14%
6	ES010	04	4.0	100.40%	3.5	102.23%	2.5	100.14%
6	ES010	05	1.0	100.36%	5.5	131.02%	5.5	127.17%
6	ES010	06	4.0	100.40%	1.5	100.67%	2.5	100.14%
7	ES010	01	3.0	99.99%	2.5	99.00%	3.0	99.26%
7	ES010	02	3.0	99.99%	5.5	112.60%	3.0	99.26%
7	ES010	03	3.0	99.99%	2.5	99.00%	3.0	99.26%
7	ES010	04	3.0	99.99%	2.5	99.00%	3.0	99.26%
7	ES010	05	6.0	100.03%	5.5	112.60%	6.0	99.27%
7	ES010	06	3.0	99.99%	2.5	99.00%	3.0	99.26%
8	ES010	01	3.0	98.15%	4.0	101.29%	3.0	100.14%
8	ES010	02	3.0	98.15%	4.0	101.29%	3.0	100.14%
8	ES010	03	3.0	98.15%	4.0	101.29%	3.0	100.14%
8	ES010	04	3.0	98.15%	4.0	101.29%	3.0	100.14%
8	ES010	05	6.0	98.67%	1.0	100.88%	6.0	100.27%
8	ES010	06	3.0	98.15%	4.0	101.29%	3.0	100.14%
9	ES010	01	3.0	99.77%	2.5	100.34%	2.0	100.46%
9	ES010	02	3.0	99.77%	5.5	101.93%	5.0	100.56%
9	ES010	03	3.0	99.77%	2.5	100.34%	5.0	100.56%
9	ES010	04	3.0	99.77%	2.5	100.34%	1.0	100.46%
9	ES010	05	6.0	99.75%	5.5	101.93%	3.0	100.68%
9	ES010	06	3.0	99.77%	2.5	100.34%	5.0	100.56%
10	ES010	01	3.0	100.33%	3.0	100.57%	3.0	99.83%
10	ES010	02	3.0	100.33%	3.0	100.57%	3.0	99.83%
10	ES010	03	3.0	100.33%	3.0	100.57%	3.0	99.83%
10	ES010	04	3.0	100.33%	3.0	100.57%	3.0	99.83%
10	ES010	05	6.0	100.70%	6.0	100.59%	6.0	99.88%
10	ES010	06	3.0	100.33%	3.0	100.57%	3.0	99.83%

$$r_j^w \ \text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right) OFI_{0,j}$$

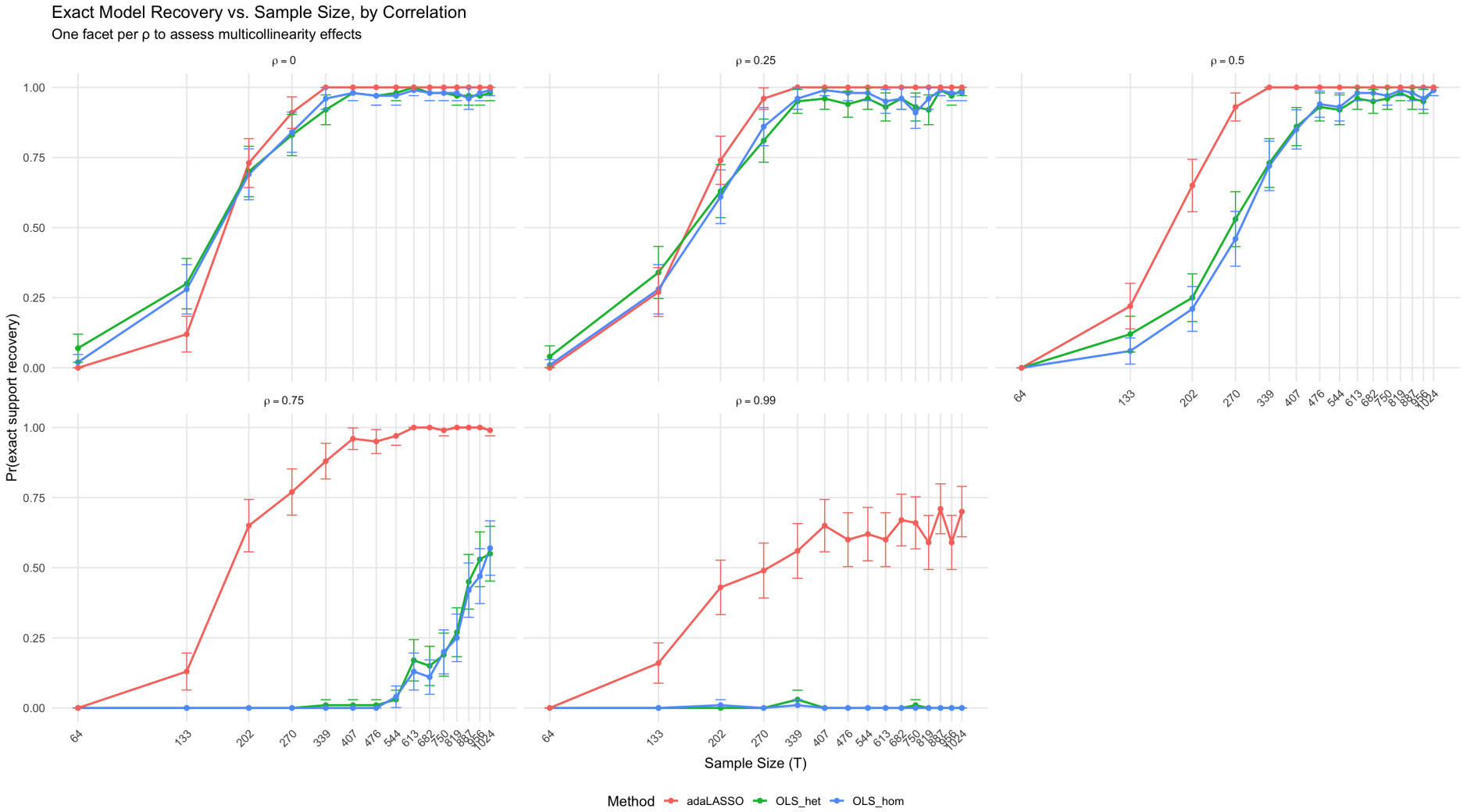


Figure 4: Probability of exact support recovery versus sample size (T) for adaptive LASSO (red) and OLS-Bonferroni (blue/green), under varying predictor correlation ρ. Each panel corresponds to a different correlation level among predictors (from $\rho = 0$ no correlation, up to $\rho = 0.99$ extreme collinearity). Error bars (± 1.96 SE) are shown for the Monte Carlo estimates.

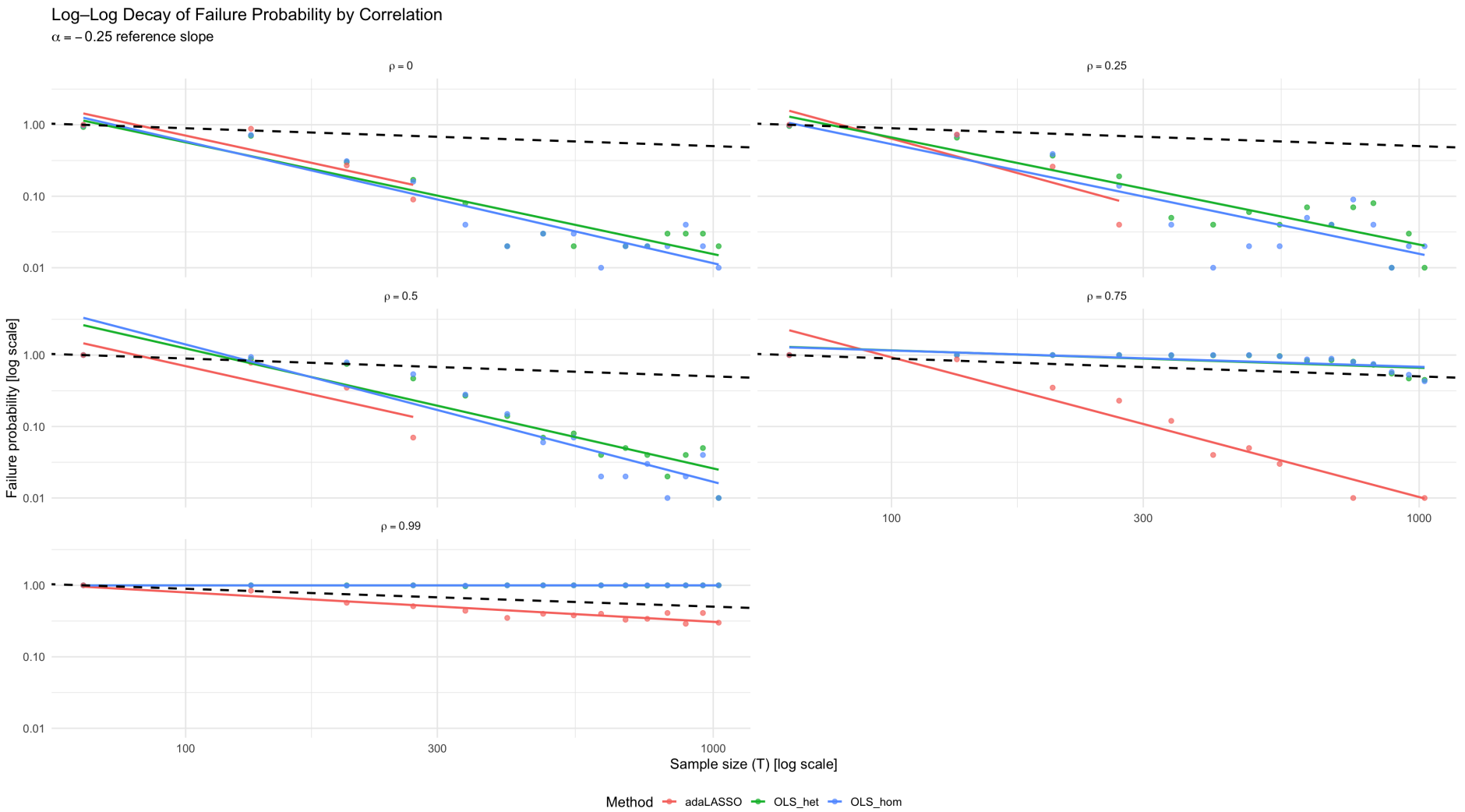


Figure 5: Log–log plot of support recovery failure probability ($\mathbb{P}(\text{sign}(\hat{\boldsymbol{\theta}}) \neq \text{sign}(\boldsymbol{\theta}))$) versus sample size T, for different correlation levels. Each panel shows the decay of failure probability as T increases on a log–log scale. The red points/line correspond to adaLASSO; blue/green to OLS (homoskedastic or robust OLS, which behave similarly). The black dashed line in each panel indicates a reference slope of -0.25 (a power-law $T^{-0.25}$) as a guide.

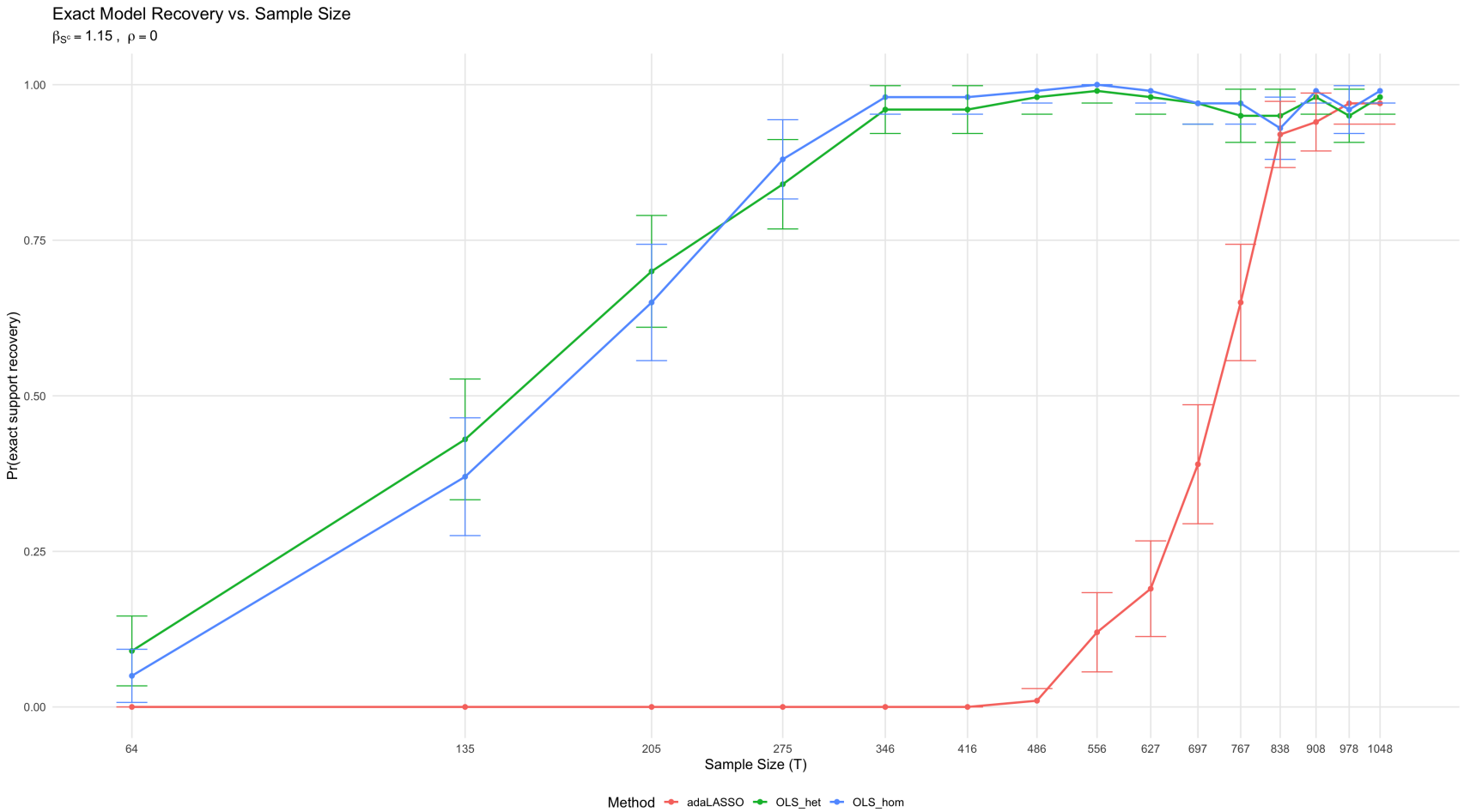


Figure 6: Probability of exact support recovery versus sample size (T) for adaptive LASSO (red) and OLS-Bonferroni (blue/green), under uncorrelated covariates and a comparatively slower discrimination rate $\beta_{S^c} = 1.15$.

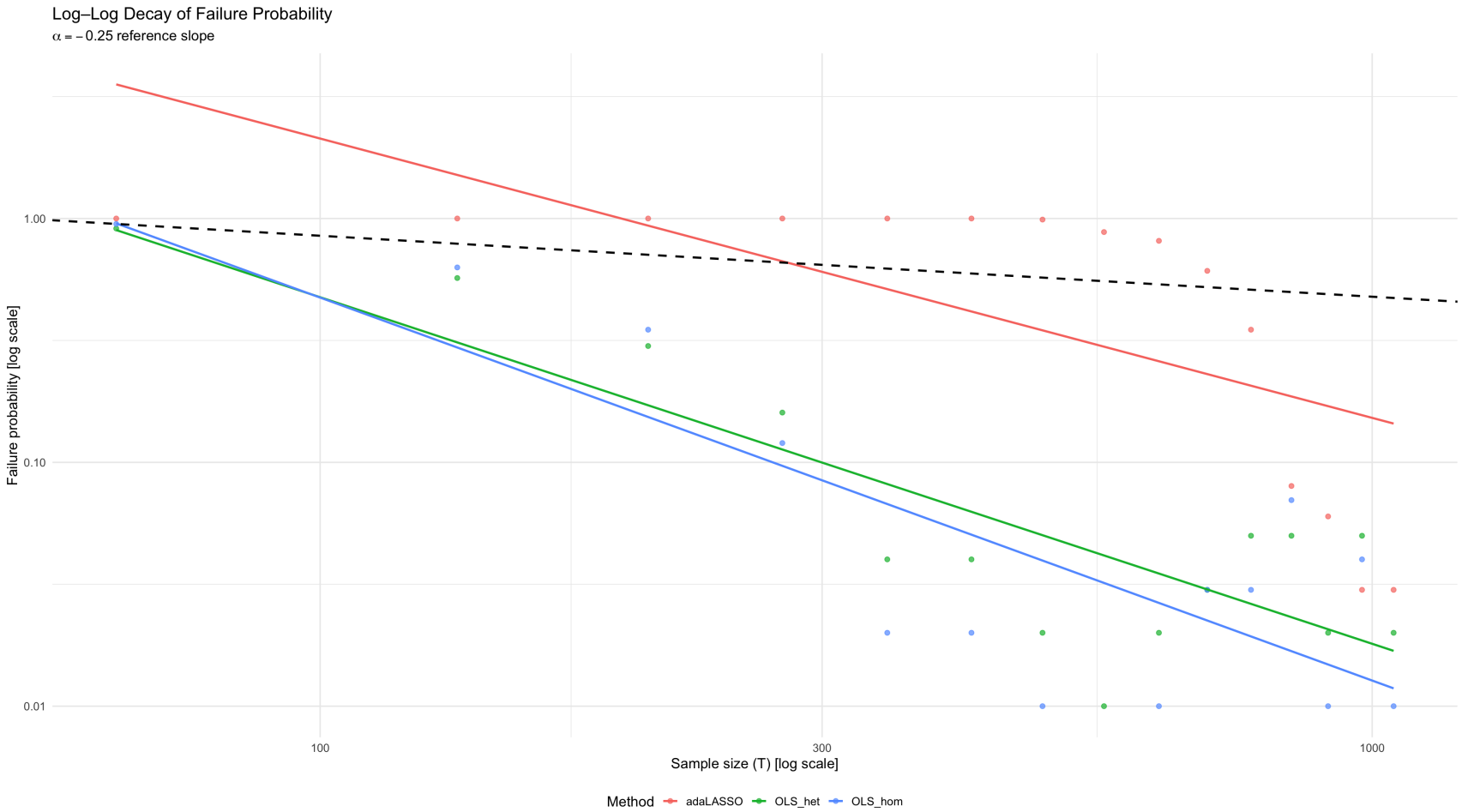


Figure 7: Log–log plot of support recovery failure probability ($\mathbb{P}(\text{sign}(\hat{\boldsymbol{\theta}}) \neq \text{sign}(\boldsymbol{\theta}))$) versus sample size T, assuming uncorrelated variables and a comparatively slower discrimination rate $\beta_{S^c} = 1.15$.

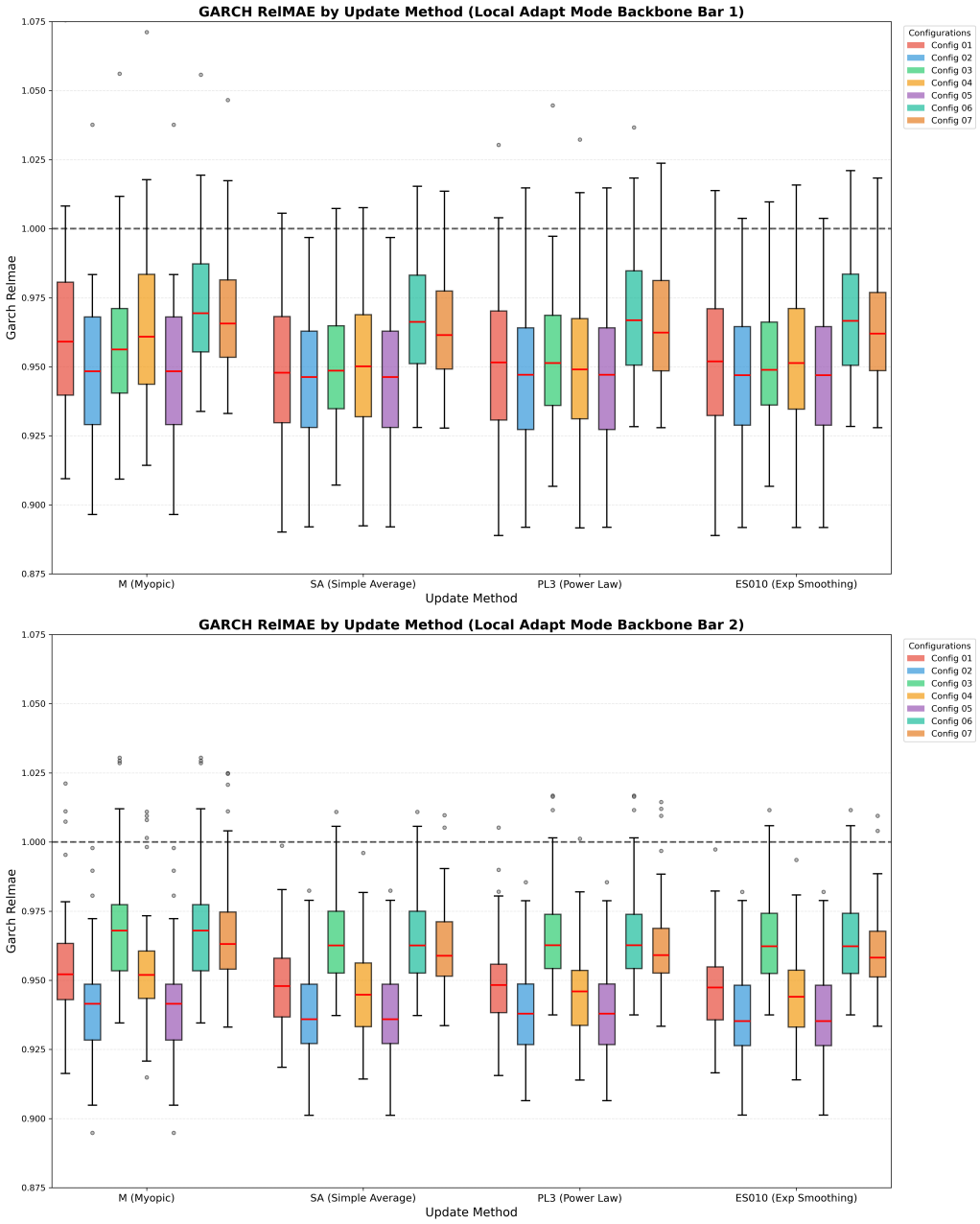


Figure 8: GARCH RelMAE comparison across update methods and configurations.

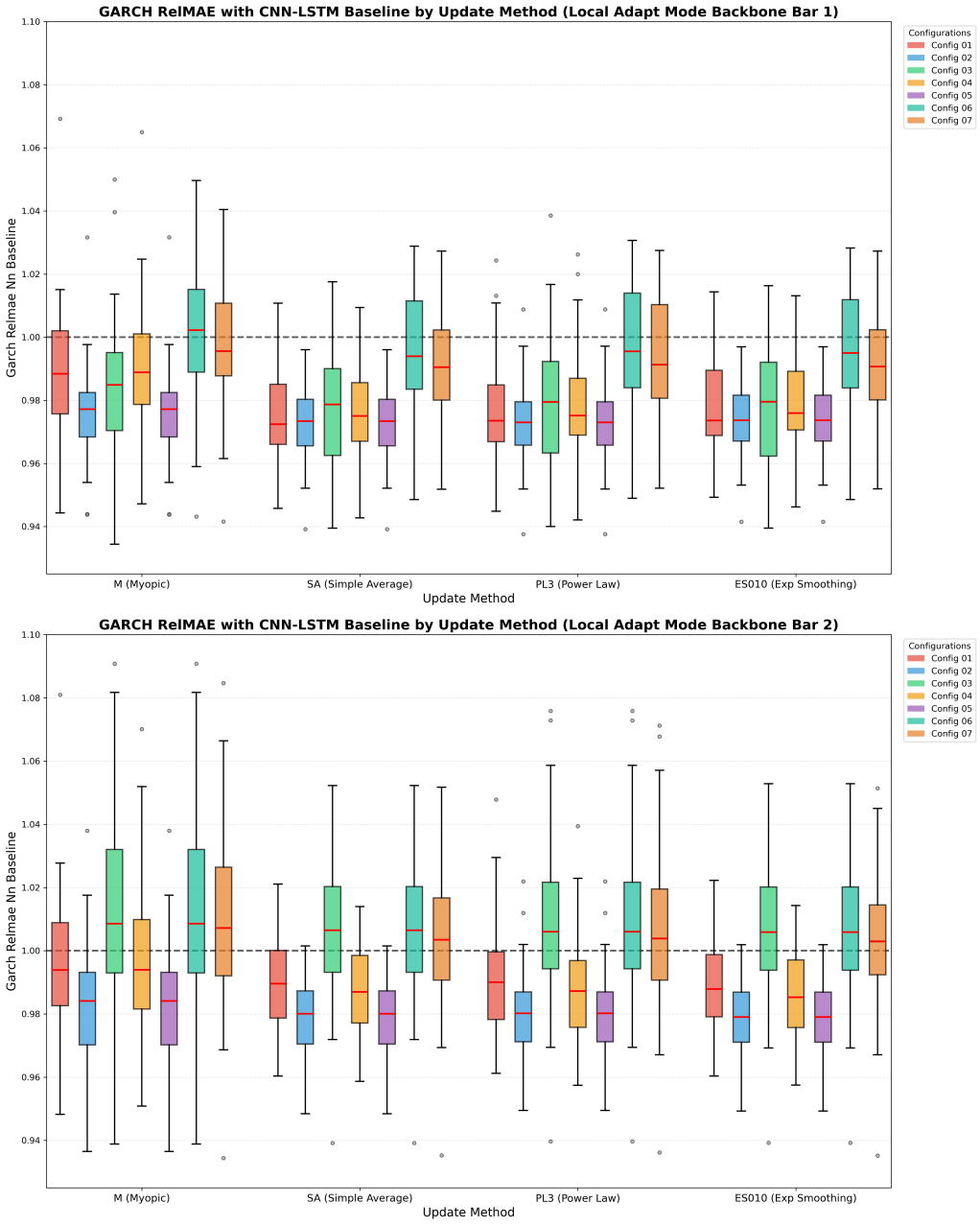


Figure 9: GARCH RelMAE comparison across update methods and configurations with CNN-LSTM baseline.

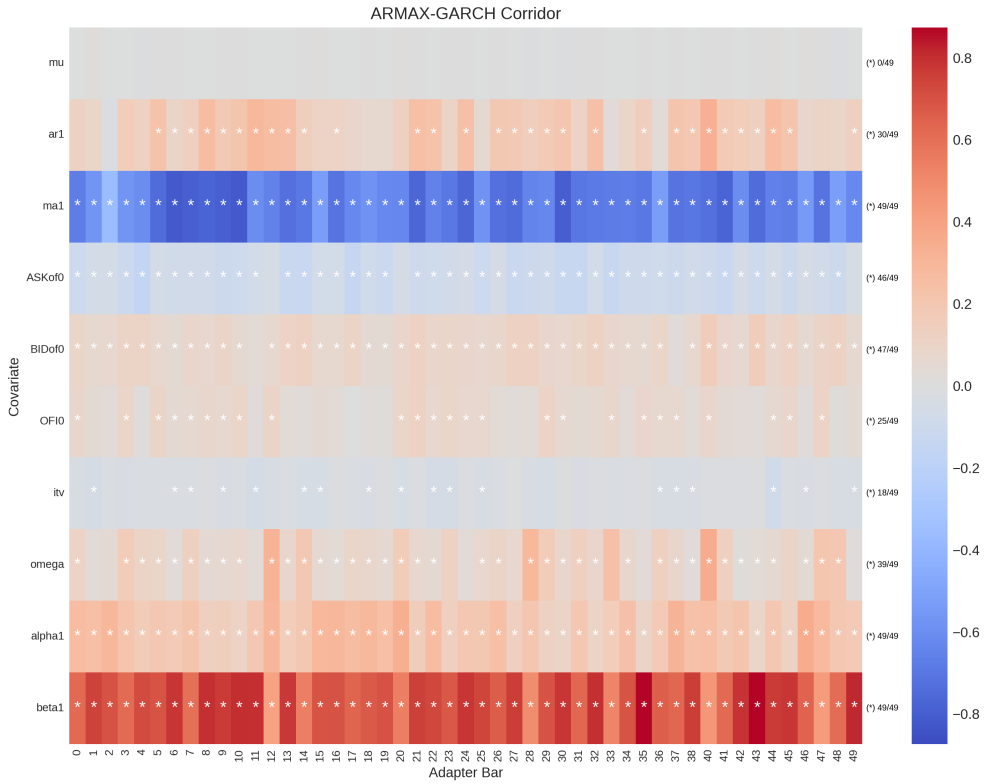


Figure 10: ARMAX–GARCH-DeepLOB corridor for configuration `SA_03`. Cells show bar-specific coefficient magnitudes by adapter bar; asterisks mark statistical significance.

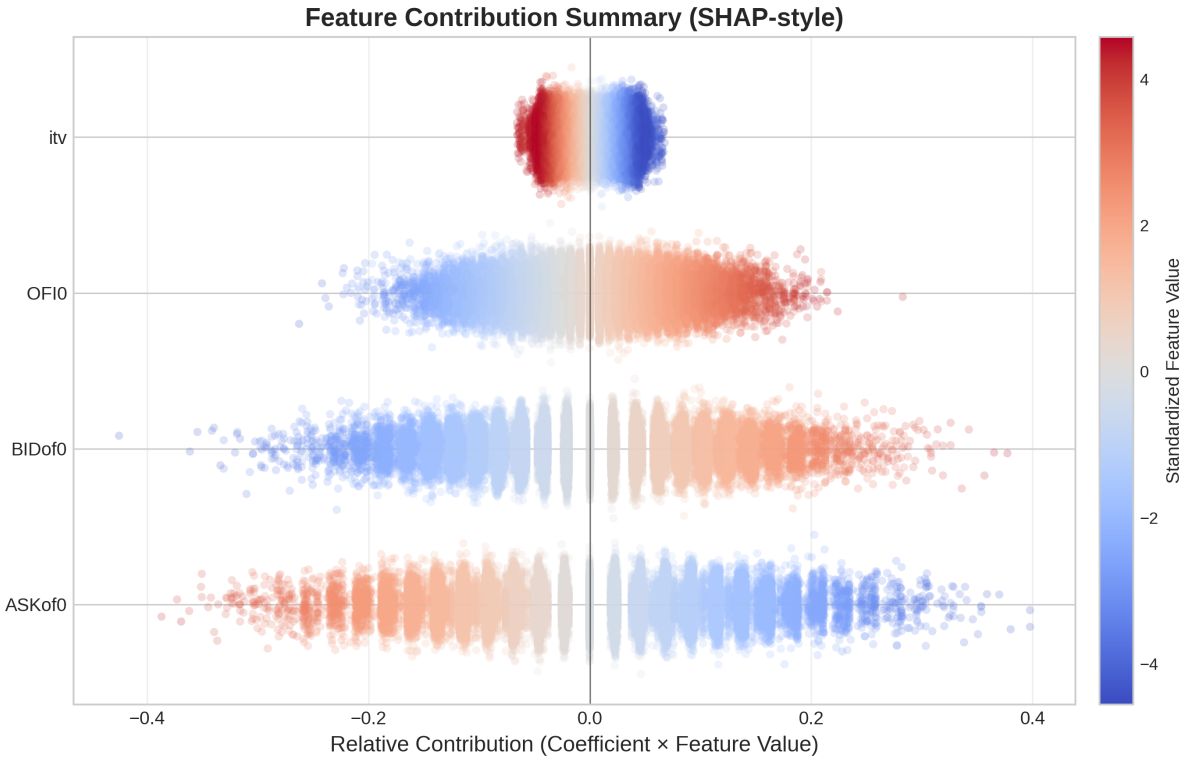


Figure 11: ARMAX–GARCH-DeepLOB SHAP scores `SA_03`. The figure plots the product of the coefficient and the feature value for each sample.



Figure 12: Evolution of parameter estimates for configuration `SA_03`. Blue lines denote combined estimates, while red dashed lines denote bar-specific estimates across adapter bars.

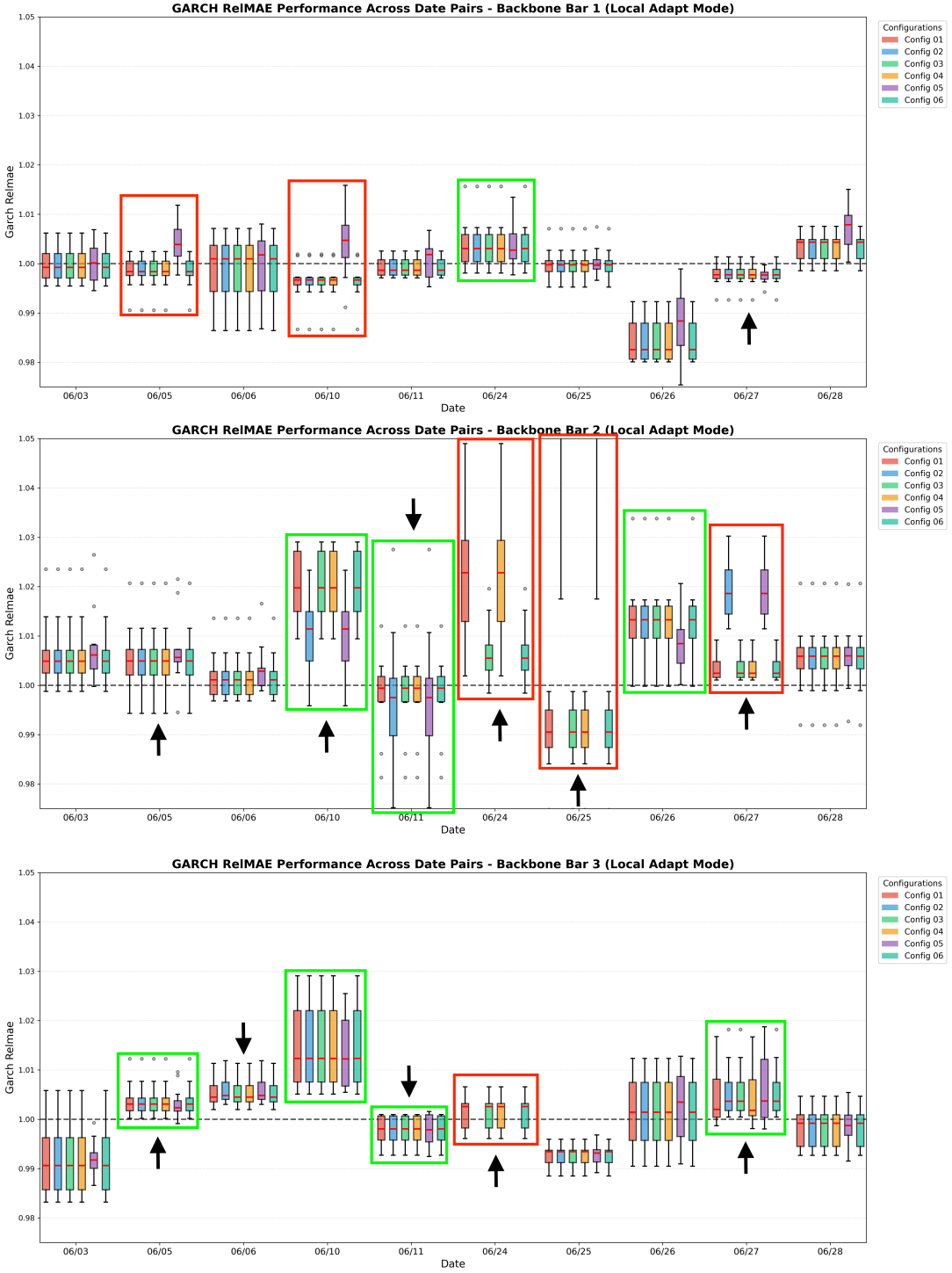


Figure 13: Sequential RelMAE boxplots for the INTC dataset across date pairs in Local Adapt mode for Backbone Bar 1 (top), Backbone Bar 2 (middle), and Backbone Bar 3 (bottom). The horizontal dashed line at 1.00 represents the naive model baseline. Green rectangles highlight instances where covariate configurations `05` (raw CNN-LSTM output) or `01` (full configuration) outperformed configuration `06` (no covariates). Red rectangles indicate date pairs where configuration `05` exhibited significant underperformance. Black arrows mark backbone bars where the CNN-LSTM weights were updated during the preceding Global Retrain step. Note that the boxplots corresponding to configurations `02` and `05` of Backbone Bars 2 and 3 on 06/24, and Backbone Bar 2 on 06/27, lie above the superior margin.

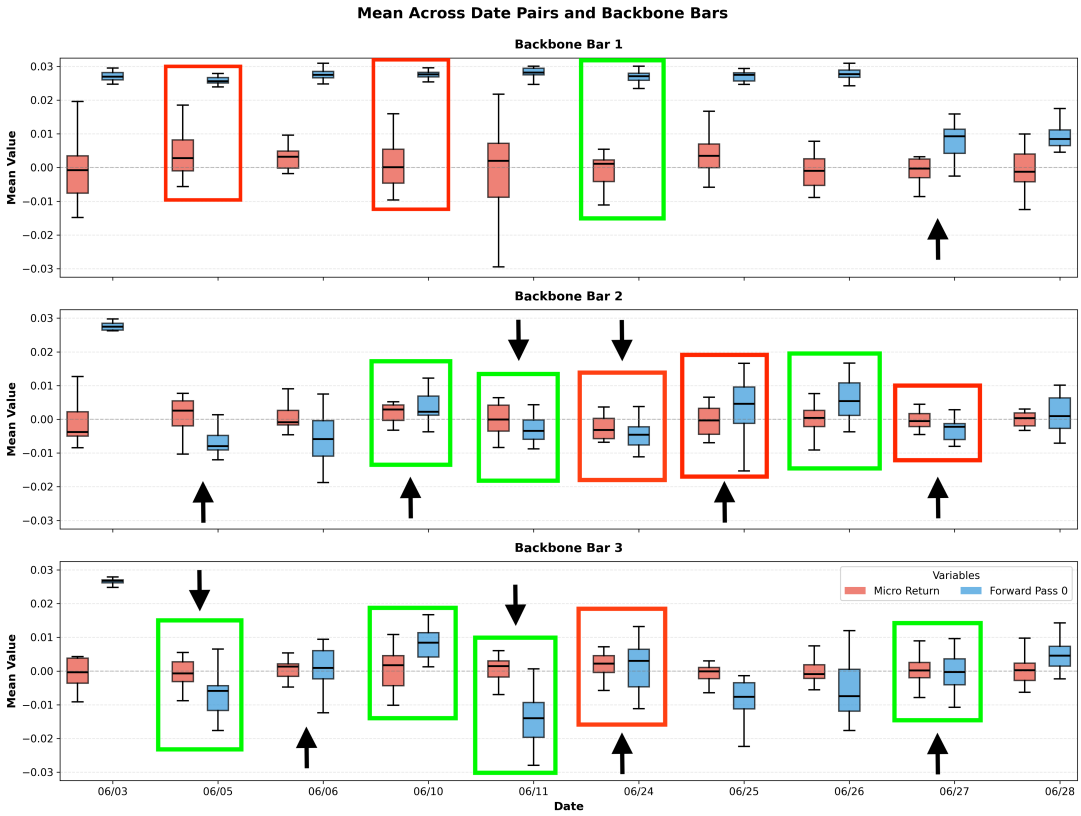


Figure 14: Distribution of the **mean** of the Micro-Return (r_j^w) and CNN-LSTM Output ($\text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right)$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

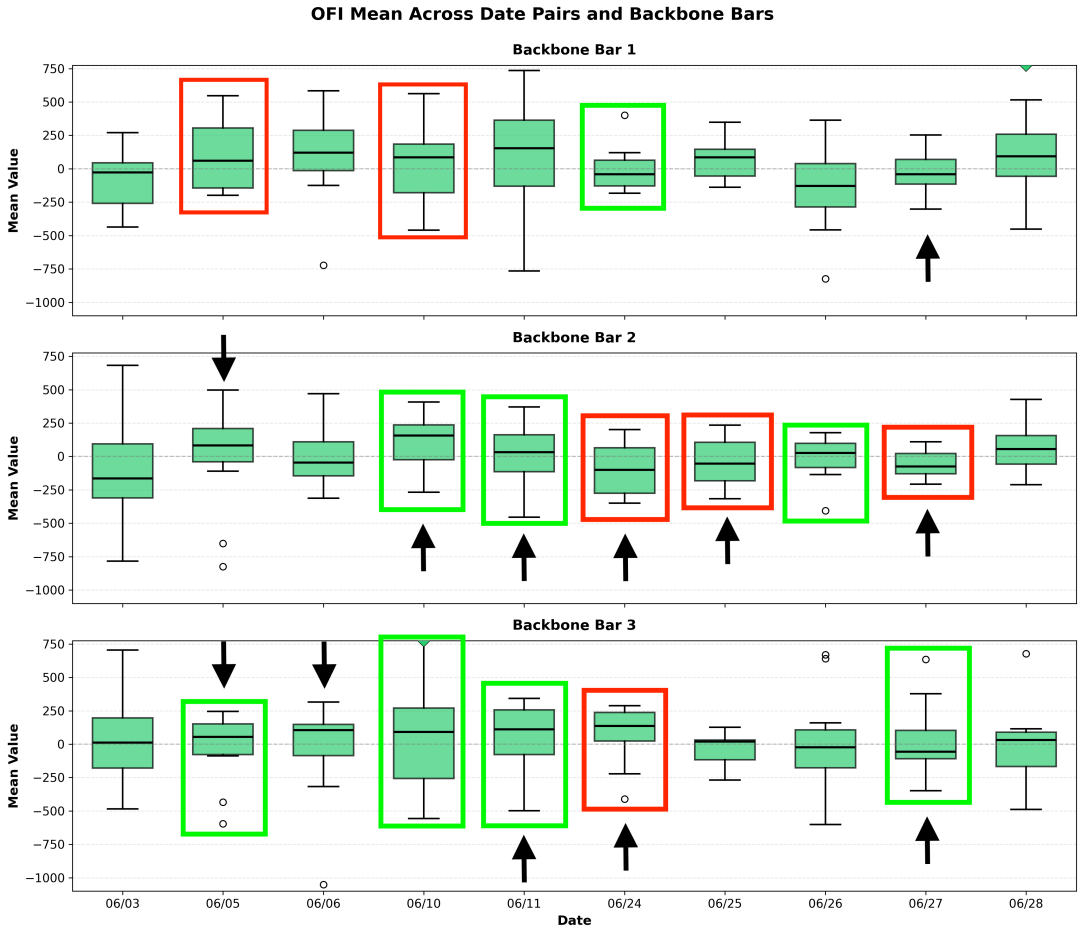


Figure 15: Distribution of the **mean** of the Order Flow Imbalance at touch ($OFI_{0,j}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

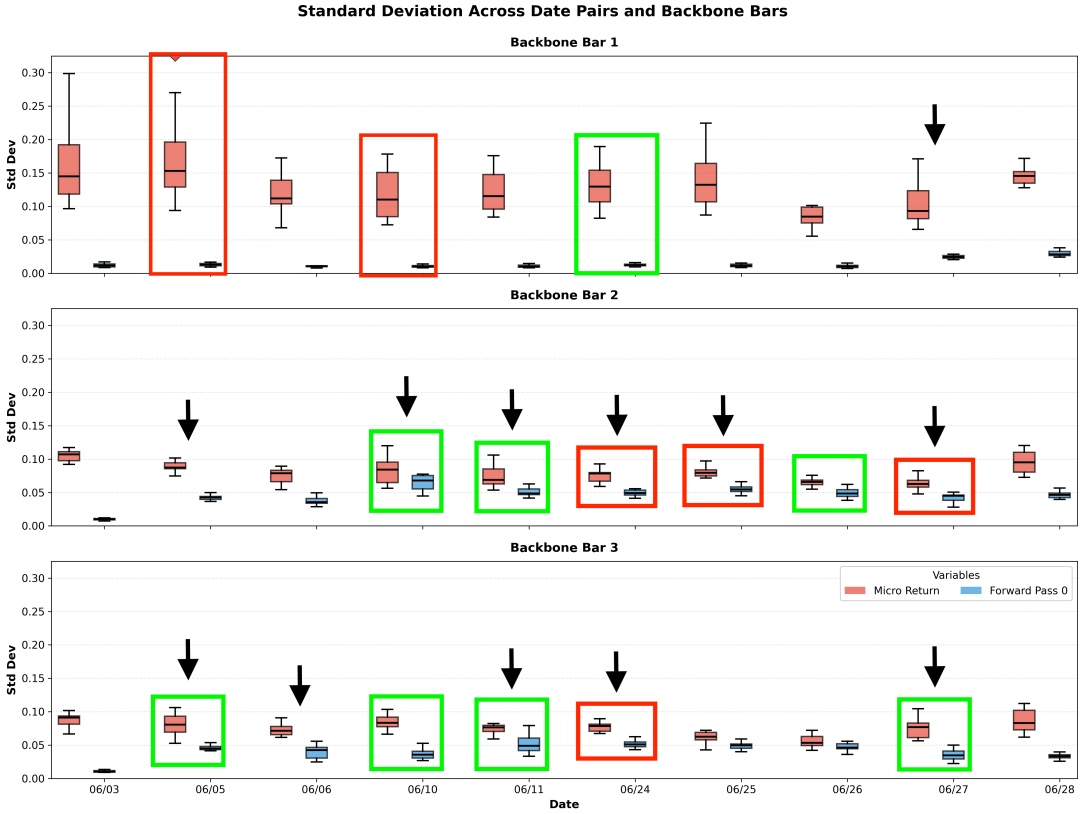


Figure 16: Distribution of the **standard deviation** of the Micro-Return (r_j^w) and CNN-LSTM Output ($\text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right)$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

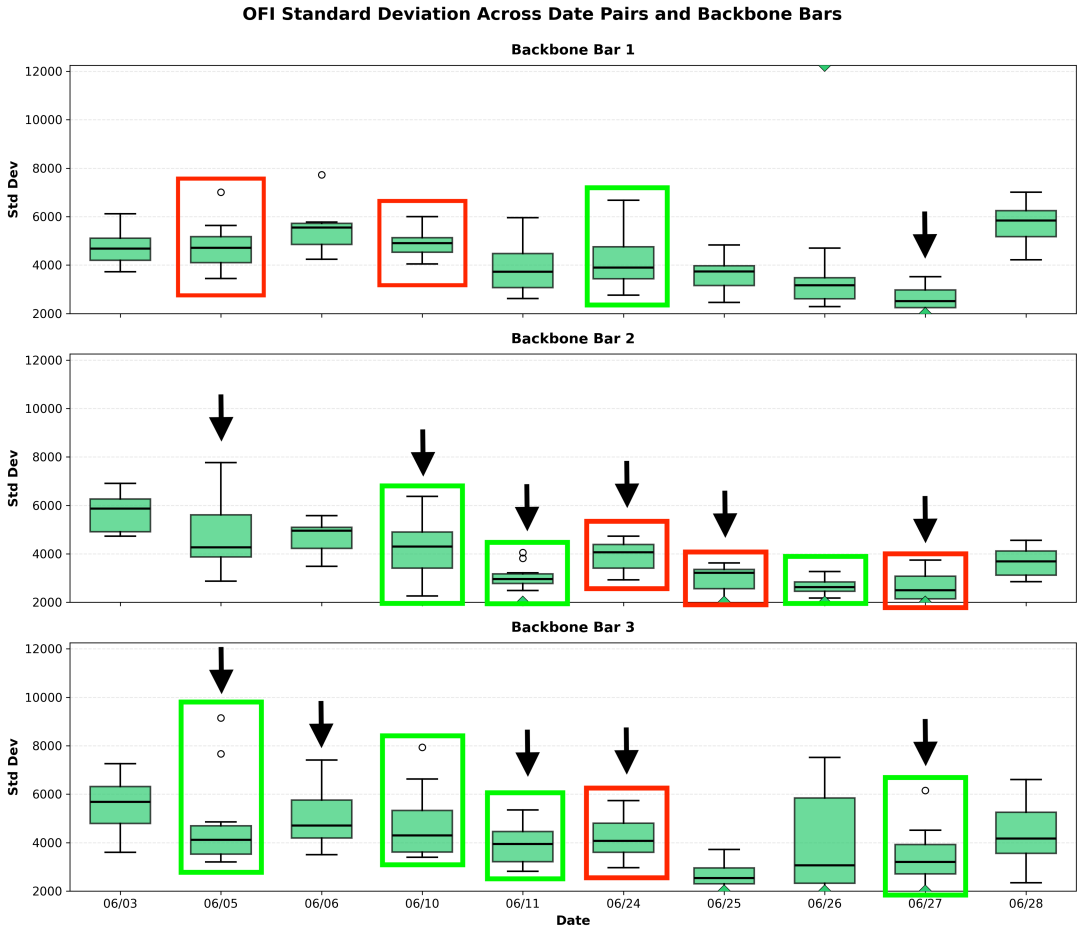


Figure 17: Distribution of the **standard deviation** of the Order Flow Imbalance at touch ($OFI_{0,j}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

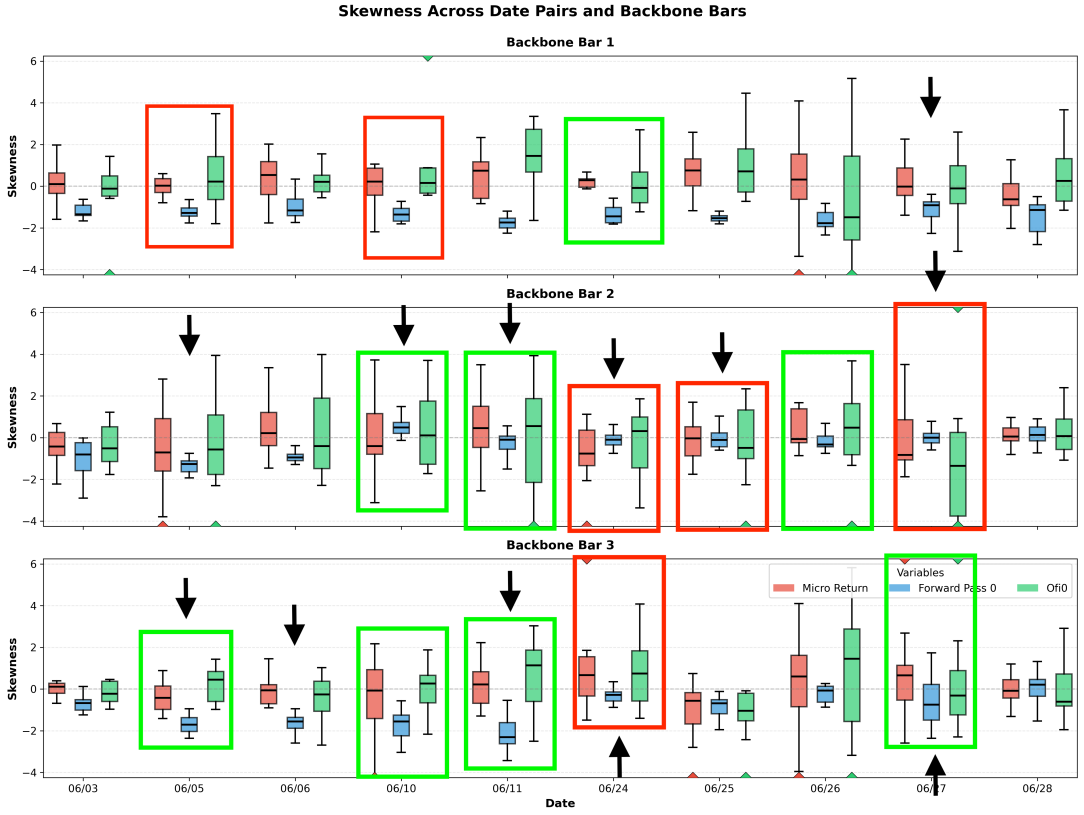


Figure 18: Distribution of the **skewness** of the Micro-Return (r_j^w), CNN-LSTM Output ($\text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right)$), and Order Flow Imbalance at touch ($OFI_{0,j}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

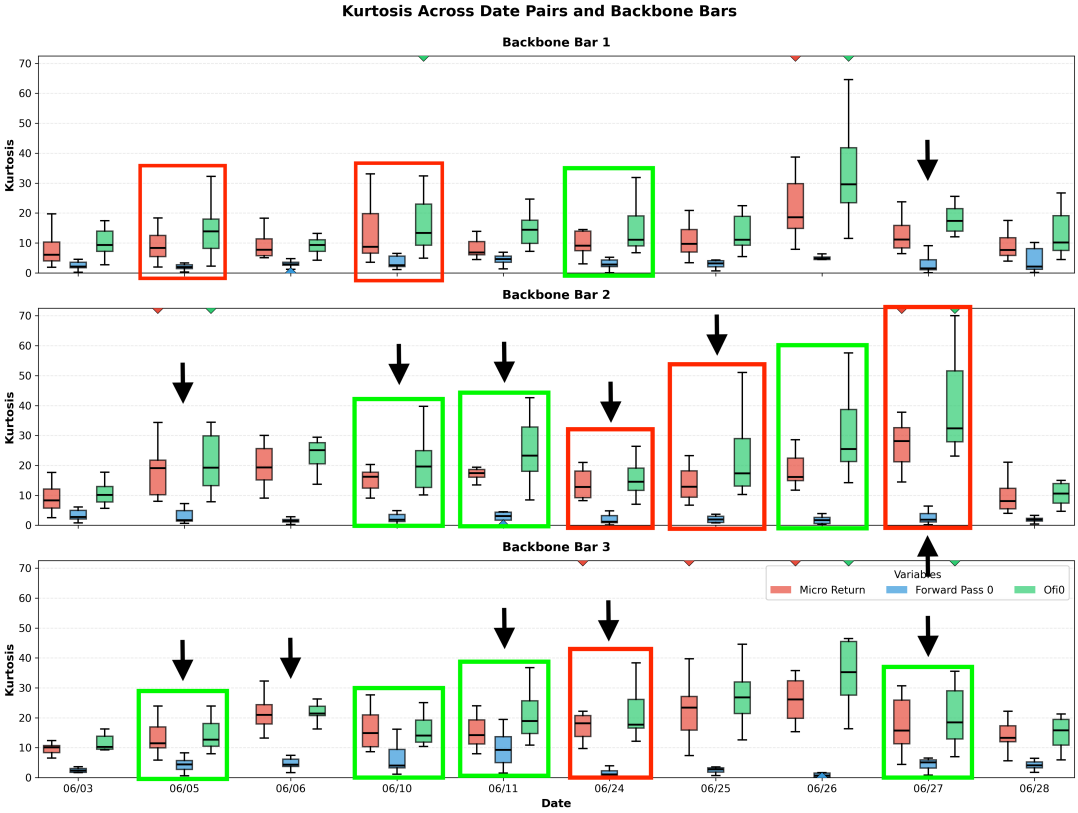


Figure 19: Distribution of the **kurtosis** of the Micro-Return (r_j^w), CNN-LSTM Output ($\text{CNNLSTM}_{\mathbf{W}}\left(\widetilde{\mathbf{OF}}^{\mathcal{P}^{(E)}}_{i_{t-1}-99:i_{t-1}}\right)$), and Order Flow Imbalance at touch ($OFI_{0,j}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

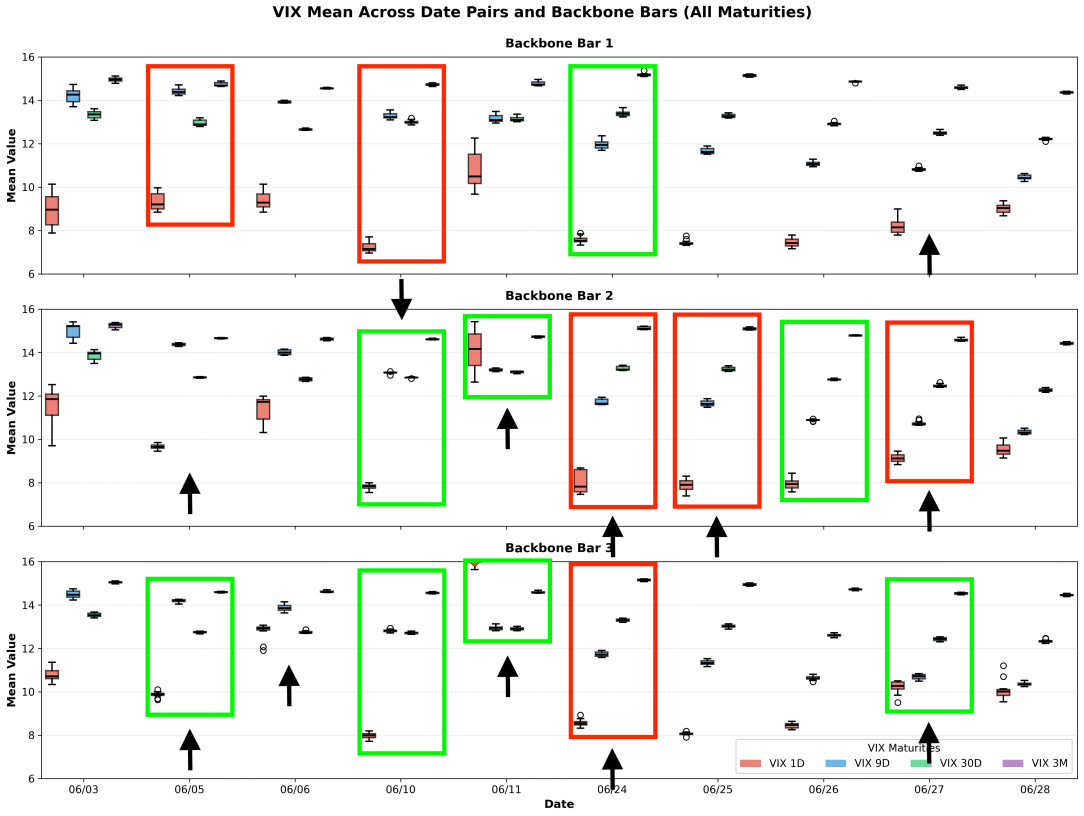


Figure 20: Distribution of the **mean** of the VIX indices ($VIX_t^{(\tau)}$ for $\tau \in \{1\text{d}, 9\text{d}, 30\text{d}, 3\text{m}\}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

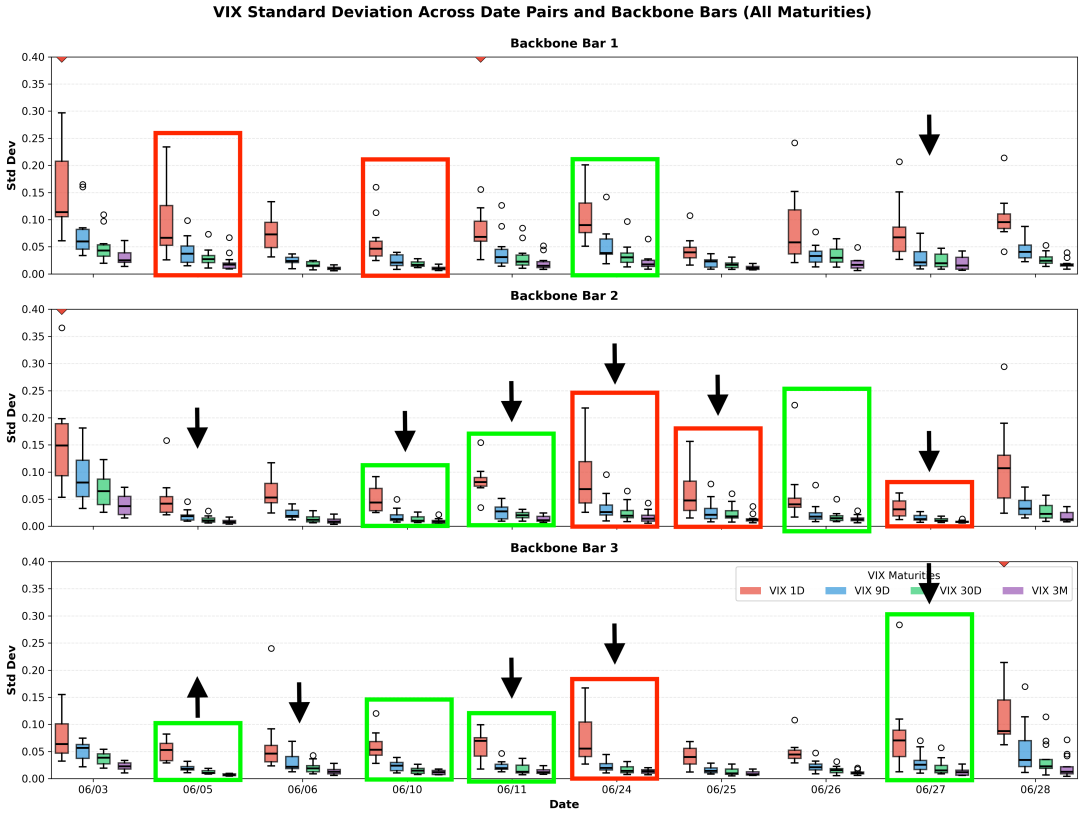


Figure 21: Distribution of the **standard deviation** of the VIX indices ($VIX_t^{(\tau)}$ for $\tau \in \{1\text{d}, 9\text{d}, 30\text{d}, 3\text{m}\}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

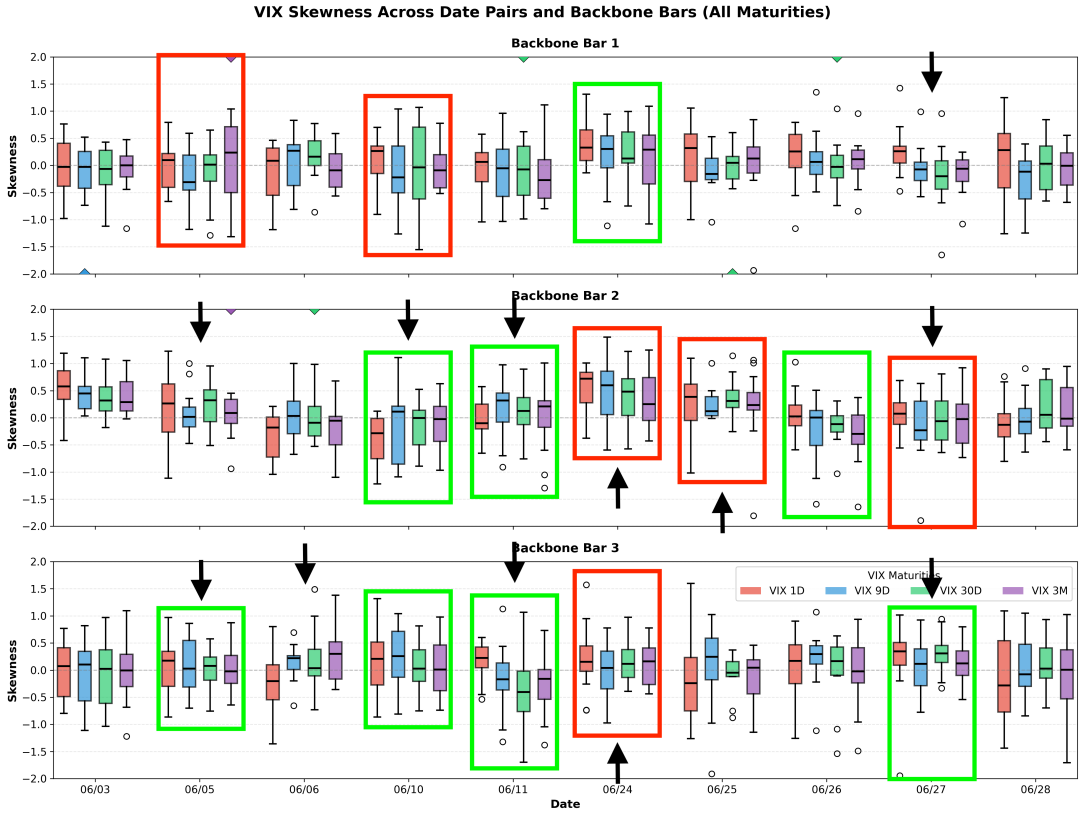


Figure 22: Distribution of the **skewness** of the VIX indices ($VIX_t^{(\tau)}$ for $\tau \in \{1\text{d}, 9\text{d}, 30\text{d}, 3\text{m}\}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

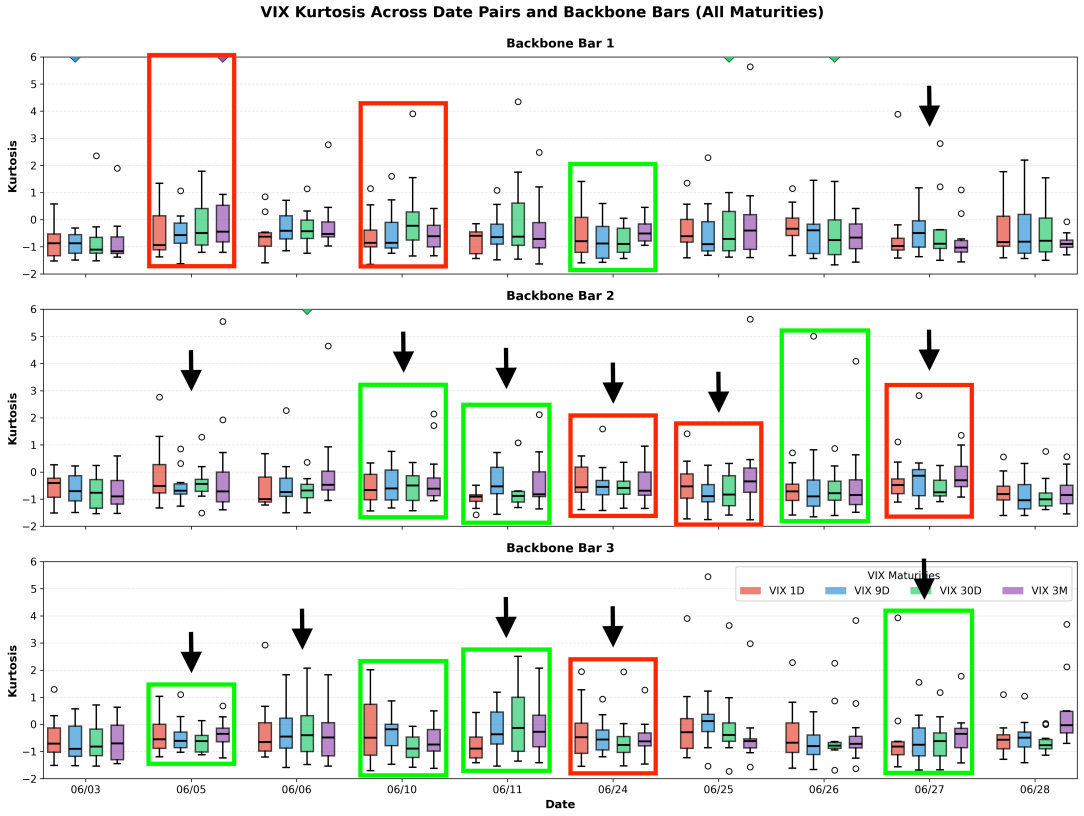


Figure 23: Distribution of the **kurtosis** of the VIX indices ($VIX_t^{(\tau)}$ for $\tau \in \{1\text{d}, 9\text{d}, 30\text{d}, 3\text{m}\}$) computed on each adapter bar and aggregated at the backbone bar level. See Figure **??** for the interpretation of the markers.

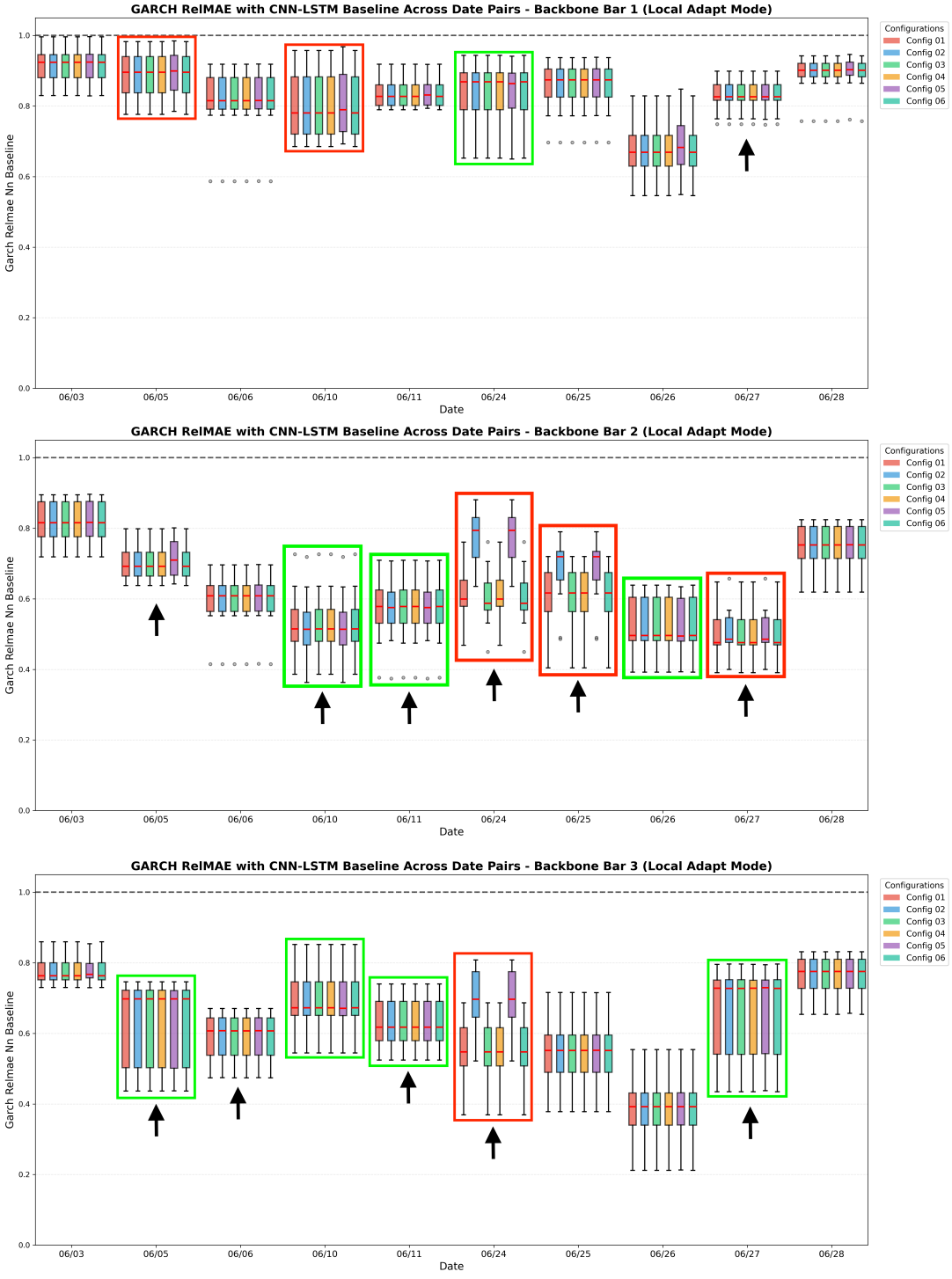


Figure 24: Sequential RelMAE boxplots for the INTC dataset across date pairs in Local Adapt mode using the **raw CNN-LSTM imputation as the baseline** (dashed line at 1.00). Top: Backbone Bar 1; Middle: Backbone Bar 2; Bottom: Backbone Bar 3. See Figure **??** for the interpretation of the markers (arrows and rectangles).